

Supplementary Information

A flexible modelling framework for estimating thermal sensitivity across life

Daniel W.A. Noble^{*1}, Pieter A. Arnold¹, Patrice Pottier²

¹*Division of Ecology and Evolution, Research School of Biology, Australian National University, Canberra, Australia*

²*Department of Biological and Environmental Sciences, University of Gothenburg, Gothenburg, Sweden*

2026-05-13

Table of contents

1	Introduction	2
1.1	<i>What each function does</i>	2
2	A Tutorial with Simulations	2
2.0.1	<i>Simulating Data</i>	4
2.0.2	<i>The classical two-stage TDT pipeline</i>	7
2.0.3	<i>A joint Bayesian 4PL</i>	9
2.0.4	<i>Deriving z, $CT_{max_{1hr}}$, and T_{crit} from the posterior</i>	17
2.0.5	<i>Comparing the two pipelines</i>	21
2.0.6	<i>A worked example: temperature effects on every shape parameter</i>	21
2.0.7	<i>Heat injury accumulation and survival under a fluctuating trace</i>	30
2.0.8	<i>Validation: recovering known planted doses</i>	33
3	Case Study 1: Shrimp data	34
3.1	<i>Data and standardisation</i>	34
3.2	<i>Fitting the joint Bayesian 4PL</i>	35
3.2.1	<i>Sampling diagnostics</i>	35
3.2.2	<i>Posterior parameters on the natural scale</i>	35
3.3	<i>Survival curves with observed overlay</i>	35
3.4	<i>TDT landscape</i>	36
3.5	<i>Classical TDT quantities: z, $CT_{max_{1hr}}$, T_{crit}</i>	37
3.6	<i>Heat injury under three reference traces</i>	38
4	References	43
5	Session Information	43

^{*}Corresponding author: daniel.noble@anu.edu.au

1 Introduction

This supplement walks through the joint Bayesian 4-parameter logistic (4PL) framework from the main manuscript using the **bayesTLS** R package, which ships with this project. It provides functions to fit the 4PL, extract the classical TLS quantities (z , $CT_{max_{1hr}}$, T_{crit}) with full posterior uncertainty, and use them to predict heat-injury accumulation and survival fraction under arbitrary temperature time-series, optionally with a Sharpe-Schoolfield repair kernel.

Install once from GitHub, then load:

```
# install.packages("remotes") # if not already installed
remotes::install_github("danielnoble/tls_model_equivalence")
```

```
library(bayesTLS)
```

The functions chain together as a single pipeline:

1. **Standardise** raw experimental data to a common column schema (`temp`, `duration`, `n_total`, `n_surv`, ...).
2. **Fit** the joint Bayesian 4PL with the built-in `beta_binomial(link = "identity")` family and asymptote-bounded reparameterisation.
3. **Extract** z , $CT_{max_{1hr}}$, and T_{crit} — all with full posterior uncertainty.
4. **Predict** survival and heat-injury accumulation under fluctuating field temperature traces, with optional repair.
5. **Plot** each step with shared theming so figures across the workflow read consistently.

Every exported function is documented with roxygen2 (`?fit_4pl`, etc.) and the package ships with `testthat` tests — fast unit tests plus a gated set of `brms`-fitting integration tests that assert recovery of known simulation truths. R CMD `check` runs cleanly. The remainder of the supplement first walks the pipeline through a controlled simulation where the truth is known (§ A Tutorial with Simulations), then applies the same machinery end-to-end to a real lethal-TDT dataset on brown shrimp (§ Case Study 1: Shrimp data).

1.1 What each function does

Table S1 lists every exported function in the library, grouped by pipeline stage. Each function is single-purpose; the full roxygen2 documentation (with runnable `@examples`) lives in the corresponding R file.

2 A Tutorial with Simulations

This section is a guided tutorial using simulated data. We show, step by step, how the Bayesian hierarchical 4PL — under both binomial sampling and the more realistic beta-binomial (overdispersed) case — can recover thermal sensitivity (z) and $CT_{max_{1hr}}$ from the single joint model, yet provide substantial downstream benefits in terms of propagating uncertainty and giving more flexibility. We then complement the simulation with a few case studies which walk through real data that is presented within the main manuscript in other sections.

Table S1: Functions exported by the TDT analysis library, grouped by pipeline stage. Each function is documented with roxygen2 in its R file; runnable examples accompany every function.

Stage	Function	Purpose
Data	<code>standardize_data()</code>	Rename raw columns to project schema; attach metadata.
Model spec	<code>make_4pl_priors()</code>	Weakly informative priors for the disjoint-bounds 4PL.
Model spec	<code>make_4pl_formula()</code>	brms non-linear formula; identity link, all four sub-parameters.
Fit	<code>fit_4pl()</code>	Joint Bayesian 4PL fit; returns a <code>tdt_4pl_workflow</code> object.
Diagnostics	<code>diagnose_tdt_fit()</code>	Rhat / ESS / divergences / treedepth, with pass-fail flags.
Diagnostics	<code>tdt_parameter_table()</code>	Posterior summary on the natural scale (low, up, k, mid).
Predictions	<code>predict_survival_curves()</code>	Survival curves on a temp x duration grid with credible intervals.
Predictions	<code>derive_tdt_landscape()</code>	Dense 2-D survival surface for a heatmap.
TDT quantities	<code>derive_ltx_curve()</code>	Time-to-x-survival across temperature (per-draw inverse).
TDT quantities	<code>derive_temperature_for_duration()</code>	Temperature at survival = X for a fixed exposure duration.
TDT quantities	<code>derive_tdt_parameters()</code>	Per-draw log-linear regression of LT_x on temperature.
TDT quantities	<code>extract_tdt()</code>	Bundled wrapper: z, CTmax, T_crit posteriors in one object.
Heat injury	<code>make_temperature_scenarios()</code>	Three reference traces (flat / single spike / multi-spike).
Heat injury	<code>planted_dose_from_trace()</code>	Analytical HI under known z, CTmax, T_c (validation).
Heat injury	<code>predict_heat_injury()</code>	Posterior HI and survival under a temperature trace.
Heat injury	<code>repair_rate_schoolfield()</code>	Sharpe-Schoolfield repair TPC (Kelvin internally).
Plotting	<code>plot_survival_curves()</code>	Posterior curves with observed proportion overlay.
Plotting	<code>plot_ltx_curve()</code>	LT_x curve on a log-time axis (classical TDT view).
Plotting	<code>plot_tdt_landscape()</code>	Survival heatmap with contour overlay.
Plotting	<code>plot_temperature_density()</code>	Posterior density for CTmax / T_crit with a 95-percentile interval.
Plotting	<code>plot_temperature_scenarios()</code>	Three traces stacked vertically.
Plotting	<code>plot_heat_injury()</code>	HI and predicted survival trajectories with CrI bands.
Plotting	<code>plot_repair_rate()</code>	Sharpe-Schoolfield TPC curve.

2.0.1 Simulating Data

2.0.1.1 Step 1 — Setting up parameters

We choose values that might be typical of a thermal mortality assay. The four 4PL parameters are the lower and upper asymptotes (ℓ, u), the slope on the $\log_{10}$ -time axis (k), and a midpoint that varies linearly with temperature: $\text{mid}(T) = \beta_0 + \beta_1(T - \bar{T})$. Each value below has a clear biological meaning.

```
true_params <- list(
  ell      = 0.03,    # 3% of individuals survive even at the longest exposures
  u        = 0.97,    # 97% are alive at very short exposures
  k        = 8,       # steepness of the survival drop on the log10(t) axis
  m_beta0  = 1.5,     # midpoint at the grand-mean temperature: ~31.6 min
  m_beta1  = -0.15,   # midpoint drops 0.15 log10-min per °C; z = -1/beta1  6.7 °C
  T_bar    = 34       # grand-mean temperature
)
```

The implied thermal sensitivity is $z = -1/\beta_1 \approx 6.7^\circ\text{C}$ — the temperature change that shifts LT50 by a factor of 10. From the same parameters we can also compute the simulation truth for the uncentred TDT intercept $\alpha = \beta_0 + \frac{1}{k} \log \frac{u-0.5}{0.5-\ell} - \beta_1 \bar{T}$ and the critical thermal limit at 1 hour $CT_{max_{1hr}} = (\log_{10} 60 - \alpha)/\beta_1$ — both approaches below will be checked against these known values.

Recall that we can calculate z , z and $CT_{max_{1hr}}$ from the true parameters using the equations in the main manuscript. We'll do that now so we have a comparator for later.

```
true_z      <- -1 / true_params$m_beta1
true_alpha  <- true_params$m_beta0 +
  (1 / true_params$k) *
  log((true_params$u - 0.5) / (0.5 - true_params$ell)) -
  true_params$m_beta1 * true_params$T_bar
true_CTmax  <- (log10(60) - true_alpha) / true_params$m_beta1

# T_crit under the rate-multiplier definition (see §[Deriving z, CTmax, and
# T_crit from the posterior]). For each posterior draw, T_crit is computed as
# CTmax_1hr + z * log10(r*/100) where r* is sampled uniformly on log10
# across the default range c(0.1, 1) % HI per hour. The MEDIAN of that
# distribution lies at log10(r*/100) = -2.5 (the geometric mean of the
# range), so:
true_T_crit <- true_CTmax - 2.5 * true_z
```

2.0.1.2 Step 2 — Simulated study design

We mimic a textbook TDT experiment: a grid of five assay temperatures crossed with six log-spaced exposure durations, repeated 30 times per cell, with 30 individuals tested per replicate. That gives 900 trials and 27,000 individuals — large enough to recover the parameters with negligible uncertainty, but small enough to fit in a few minutes. Of course, in practice we will not have

this much data as will be seen in the case studies, but simulations are a good starting point to understand the models being fit and the conversions.

```
temps      <- c(30, 32, 34, 36, 38)      # 5 assay temperatures (°C)
durations  <- c(1, 5, 15, 45, 135, 405)  # 6 durations (minutes, log-spaced)
n_rep      <- 30                         # 30 replicates per cell
n_per_rep  <- 30                         # 30 individuals per replicate

design <- tidyr::expand_grid(
  T      = temps,
  t      = durations,
  rep    = seq_len(n_rep)
) |>
  dplyr::mutate(
    log10_t = log10(t),
    T_c      = T - true_params$T_bar,      # mean-centred T
    mid      = true_params$m_beta0 + true_params$m_beta1 * T_c, # mid(T)
    p_true   = true_params$ell +
      (true_params$u - true_params$ell) /
      (1 + exp(true_params$k * (log10_t - mid))),
    n        = n_per_rep
  )
```

2.0.1.3 Step 3 — Look at the truth before simulating

Before generating noisy data, let's plot the *true* survival surface (Figure S1) so we know what the model is being asked to recover. Each line is the 4PL at one assay temperature, plotted against $\log_{10}$ exposure time.

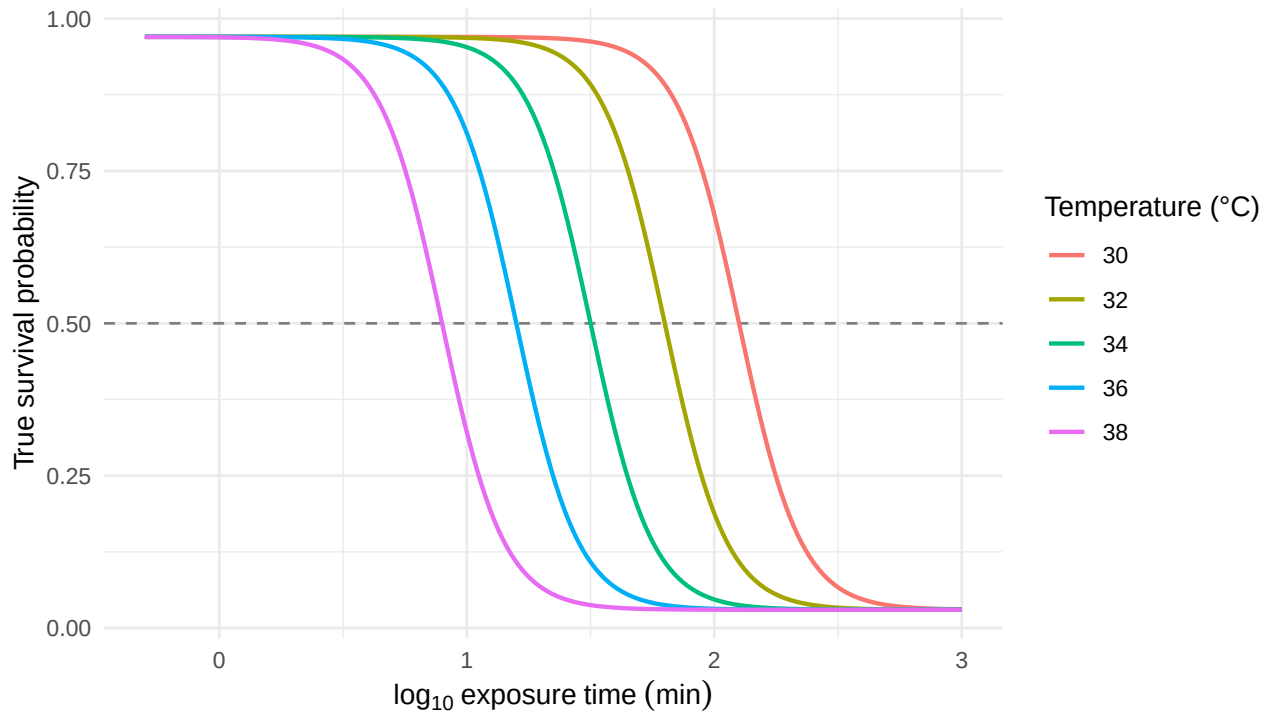


Figure S1: The known 4PL dose-response surface used to generate the simulated data, evaluated on a dense time grid at each of the five assay temperatures. The horizontal line marks 50% survival; where each curve crosses it gives the true LT50 at that temperature.

Notice the curves shift left as temperature rises: hotter temperatures kill faster, so the LT50 is reached at shorter durations. That horizontal shift in midpoints is exactly what β_1 encodes.

2.0.1.4 Step 4 — Generate noisy data: binomial and beta-binomial

Real survival counts have at least two flavours of noise. **Binomial** sampling is the textbook case: each replicate's survival count is a draw from $\text{Binomial}(n, p)$ with p from the 4PL surface. **Beta-binomial** sampling adds *overdispersion* — replicate-to-replicate variability in p that the binomial cannot capture, e.g., from cohort effects or unmodelled heterogeneity. Real TDT data almost always show overdispersion so in many cases the beta binomial will be a better fit.

```
# (a) Binomial: y ~ Binomial(n, p_true)
sim_bin <- design |>
  dplyr::mutate(y = rbinom(dplyr::n(), size = n, prob = p_true))

# (b) Beta-binomial: replicate-level p drawn from Beta(p*phi, (1-p)*phi)
phi <- 5 # smaller phi = more overdispersion
sim_bb <- design |>
  dplyr::mutate(
    p_draw = rbeta(dplyr::n(), p_true * phi, (1 - p_true) * phi),
    y       = rbinom(dplyr::n(), size = n, prob = p_draw)
  )
```

Table S2: Per-temperature Stage-1 estimates of $\log_{10}$ LT50 from a binomial GLM with logit link, fit separately at each of the five assay temperatures. Truth values are the simulated midpoints (which coincide with $\log_{10}$ LT50 here because the simulation asymptotes are near 0 and 1).

T (°C)	Truth $\log_{10}(\text{LT50})$	Binomial $\log_{10}(\text{LT50})$	Beta-binomial $\log_{10}(\text{LT50})$
30	2.1	2.067	2.092
32	1.8	1.794	1.791
34	1.5	1.491	1.498
36	1.2	1.193	1.197
38	0.9	0.921	0.939

2.0.2 The classical two-stage TDT pipeline

We first run the simulated data through the **classical two-stage TDT pipeline** (Ørsted *et al.* 2022; Rezende *et al.* 2014), which is dominant in the thermal-tolerance literature.

1. **Stage 1.** At each assay temperature, fit a logistic dose-response curve to the count data and read off a point estimate of $\log_{10}$ LT50_T.
2. **Stage 2.** Regress those Stage-1 point estimates on assay temperature with ordinary least squares, then derive $z = -1/\beta_1$ and $CT_{max_{1hr}} = (\log_{10} 60 - \alpha)/\beta_1$ from the fitted line.

2.0.2.1 Stage 1 — Dose-response curves at each temperature

We fit a binomial GLM with a logit link separately at each assay temperature. This is the most-common Stage-1 specification in the TDT literature: a two-parameter logistic with asymptotes constrained to 0% and 100%. The midpoint $\log_{10}$ LT50_T = $-\beta_0/\beta_1$ falls out of the GLM’s two coefficients.

```
fit_stage1 <- function(sim_data) {
  sim_data |>
  dplyr::group_by(T) |>
  dplyr::group_modify(function(d, key) {
    f <- glm(cbind(y, n - y) ~ log10_t, data = d, family = binomial())
    co <- coef(f)
    tibble::tibble(log10_lt50 = as.numeric(-co[1] / co[2]))
  }) |>
  dplyr::ungroup()
}

stage1_bin <- fit_stage1(sim_bin)
stage1_bb <- fit_stage1(sim_bb)
```

With the simulation’s true asymptotes near 0 and 1, the 2PL’s forced-asymptote constraint introduces negligible bias here, so each Stage-1 estimate sits close to the simulation truth.

2.0.2.2 Stage 2 — log-linear TDT regression

Stage 2 regresses Stage-1 $\log_{10}$ LT50 on assay temperature with ordinary least squares. The classical TDT quantities — $z = -1/\beta_1$ and $CT_{max_{1hr}} = (\log_{10} 60 - \alpha)/\beta_1$ — fall out of the fitted line.

```
fit_stage2 <- function(stage1) {  
  fit2 <- lm(log10_lt50 ~ T, data = stage1)  
  co    <- coef(fit2)  
  
  tibble::tibble(  
    quantity = c("z (°C)", "CTmax at 1 hr (°C)"),  
    est      = c(as.numeric(-1 / co["T"]),  
                 as.numeric((log10(60) - co["(Intercept)"]) / co["T"]))  
  )  
}  
  
ts_bin <- fit_stage2(stage1_bin)  
ts_bb  <- fit_stage2(stage1_bb)
```

Figure S2 visualises the Stage-2 fit. Each point is one Stage-1 $\log_{10}$ LT50 estimate from Table S2; the solid line is the OLS regression through them. The slope is β_1 , from which $z = -1/\beta_1$ — i.e., the temperature increase that causes LT50 to change by a factor of 10. The line's height at $\log_{10} 60 \approx 1.78$ (1 hour) is at the temperature equal to $CT_{max_{1hr}}$ — the vertical dotted line drops from this intersection straight to the temperature axis to mark it. The dashed grey line is the simulation truth.

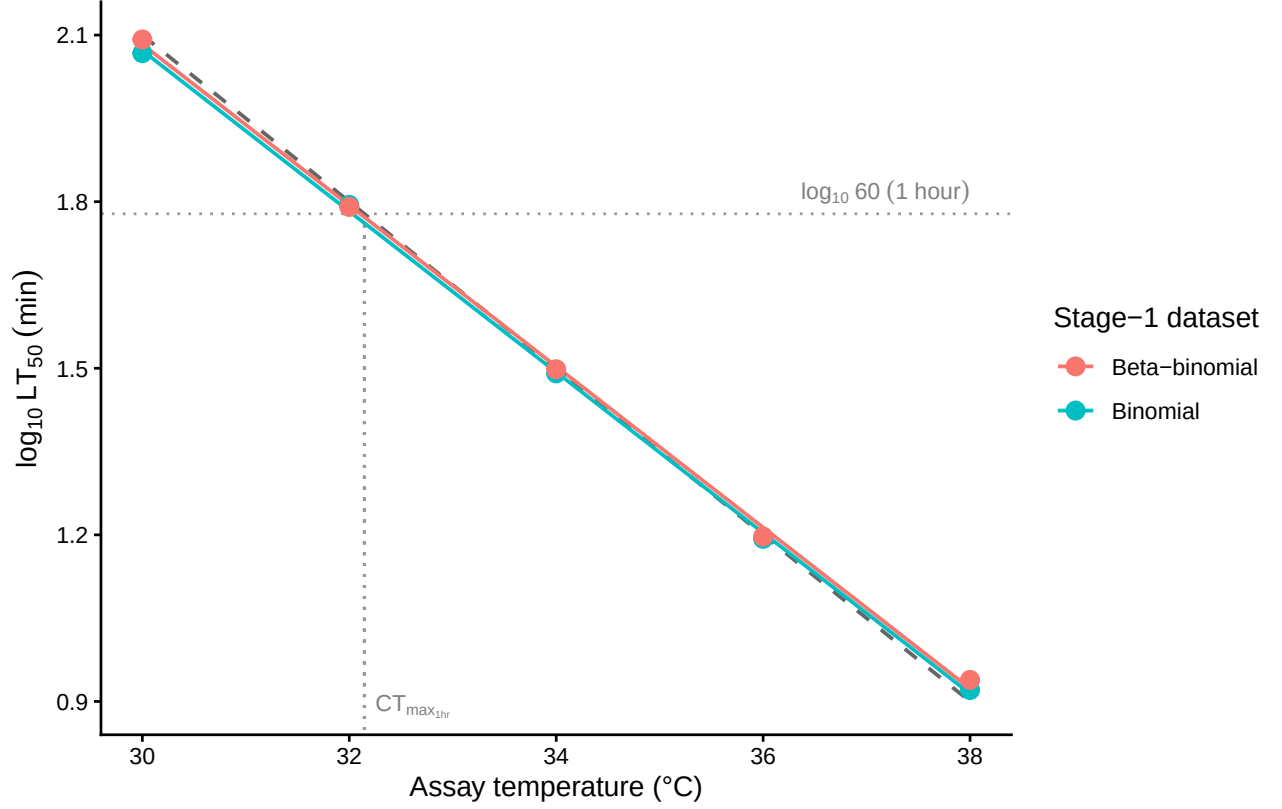


Figure S2: Stage-2 OLS regression of $\log_{10}$ LT50 on assay temperature, fit separately to the binomial and beta-binomial simulated datasets. Points are the Stage-1 per-temperature LT50 estimates; solid coloured lines are the OLS fits; the dashed grey line is the simulation truth. The slope of each line equals β_1 , from which $z = -1/\beta_1$. The horizontal dotted line marks $\log_{10} 60$ (1 hour); the vertical dotted line drops from where the truth crosses 1 hour straight down to the temperature axis, marking $CT_{max_{1hr}}$ visually.

This is the entire two-stage workflow. The point estimates that come out — z and $CT_{max_{1hr}}$ — are what the field reports. We carry them forward (`ts_bin` and `ts_bb`) for the side-by-side comparison once we have the joint Bayesian results.

! Important

Two-stage steps often ignore uncertainty in estimates in both fitting and error propagation. Uncertainty in estimates can be propagated using the SE's and the delta method, but only at each individual stage.

2.0.3 A joint Bayesian 4PL

Instead of fitting a separate logistic at each assay temperature and then regressing the LT50s, the joint fit estimates every parameter — ℓ , u , k , β_0 and β_1 — in a single hierarchical model on the raw counts. We now use the same simulated data to derive the z and $CT_{max_{1hr}}$ from the posterior, in one model rather than two.

2.0.3.1 Specifying the 4PL in brms

The model has two pieces. The first is the 4PL itself — survival probability as a function of $\log_{10}$ exposure time:

$$p_{ij} = \ell + \frac{u - \ell}{1 + \exp(k(\log_{10} t_{ij} - \text{mid}_{ij}))}, \quad (\text{S1})$$

where ℓ and u are the lower and upper asymptotes, k is the slope on the $\log_{10} t$ axis, and mid_{ij} is the value of $\log_{10} t$ at which the curve crosses the midpoint between ℓ and u . The second piece lets the midpoint vary linearly with mean-centred temperature:

$$\text{mid}_{ij} = \beta_0 + \beta_1 (T_{ij} - \bar{T}), \quad (\text{S2})$$

so that β_1 encodes how fast the dose-response curve shifts on the time axis as assay temperature changes; $\bar{T}$ is the grand-mean temperature, included so that β_0 is interpretable as the midpoint at the average assay temperature.

The function library exposes this model as a single call: `fit_4pl()` constructs the brms `bf()` formula, the default weakly-informative priors, and the sampling controls, and returns a workflow object containing the fit. Internally the formula uses the disjoint-bounds reparameterisation described in §[Loading the function library] — `low = 0.001 + \text{inv\_logit}(\text{lowraw}) \cdot 0.498` and `up = 0.501 + \text{inv\_logit}(\text{upraw}) \cdot 0.498` — so the 4PL output is guaranteed to lie in $(0,1)$ regardless of where MCMC takes the unconstrained `lowraw`, `upraw`, and `logk` parameters. The default also puts a `temp_c` slope on every 4PL sub-parameter; when temperature has no real effect on a given parameter, the slope shrinks toward zero under the prior. We inspect the brms formula `fit_4pl()` builds with:

```
make_4pl_formula()
```

```
n_surv | trials(n_total) ~ (0.001 + inv_logit(lowraw) * 0.498) + ((0.501 + inv_logit(upraw) * 0.498) * exp(logk * temp_c))
lowraw ~ temp_c
upraw ~ temp_c
logk ~ temp_c
mid ~ temp_c
```

! Important

The default model puts a `temp_c` slope on all four 4PL sub-parameters and includes no random effects. Random intercepts on `mid` can be added by passing `random_effects = c("Date", "Tank", ...)` to `fit_4pl()`. The simulation here has no batch structure so we leave it off; the shrimp case study below uses `random_effects = c("Date", "Tank")`.

2.0.3.2 Standardising and fitting

The first step is to pass the simulated data through `standardize_data()` so the columns match the schema the rest of the library expects (`temp`, `duration`, `logd`, `temp_c`, `n_total`, `n_surv`).

```
std_bin <- standardize_data(sim_bin,
                           temp = "T", duration = "t",
                           n_total = "n", n_surv = "y",
                           duration_unit = "minutes")
std_bb  <- standardize_data(sim_bb,
                           temp = "T", duration = "t",
                           n_total = "n", n_surv = "y",
                           duration_unit = "minutes")
```

We now call `fit_4pl()` twice — once with the built-in `binomial(link = "identity")` family for the non-overdispersed simulation, and once with the default `beta_binomial(link = "identity")` for the overdispersed case. Caching is handled by brms's `file = ...` mechanism, so subsequent renders reload the cached fits unless the formula or data changes.

```
wf_bin <- fit_4pl(std_bin,
                 family = binomial(link = "identity"),
                 chains = 4, iter = 2000, cores = 4, seed = 123,
                 file = file.path(models_dir, "sim_4pl_binomial_v2"))
```

Running MCMC with 4 parallel chains...

```
Chain 1 Iteration:    1 / 2000 [  0%] (Warmup)
Chain 2 Iteration:    1 / 2000 [  0%] (Warmup)
Chain 3 Iteration:    1 / 2000 [  0%] (Warmup)
Chain 4 Iteration:    1 / 2000 [  0%] (Warmup)
Chain 2 Iteration:   100 / 2000 [  5%] (Warmup)
Chain 4 Iteration:   100 / 2000 [  5%] (Warmup)
Chain 1 Iteration:   100 / 2000 [  5%] (Warmup)
Chain 3 Iteration:   100 / 2000 [  5%] (Warmup)
Chain 2 Iteration:   200 / 2000 [ 10%] (Warmup)
Chain 4 Iteration:   200 / 2000 [ 10%] (Warmup)
Chain 1 Iteration:   200 / 2000 [ 10%] (Warmup)
Chain 2 Iteration:   300 / 2000 [ 15%] (Warmup)
Chain 3 Iteration:   200 / 2000 [ 10%] (Warmup)
Chain 4 Iteration:   300 / 2000 [ 15%] (Warmup)
Chain 2 Iteration:   400 / 2000 [ 20%] (Warmup)
Chain 1 Iteration:   300 / 2000 [ 15%] (Warmup)
Chain 3 Iteration:   300 / 2000 [ 15%] (Warmup)
Chain 2 Iteration:   500 / 2000 [ 25%] (Warmup)
Chain 4 Iteration:   400 / 2000 [ 20%] (Warmup)
Chain 1 Iteration:   400 / 2000 [ 20%] (Warmup)
Chain 2 Iteration:   600 / 2000 [ 30%] (Warmup)
```

Chain 3 Iteration: 400 / 2000 [20%] (Warmup)
 Chain 4 Iteration: 500 / 2000 [25%] (Warmup)
 Chain 1 Iteration: 500 / 2000 [25%] (Warmup)
 Chain 2 Iteration: 700 / 2000 [35%] (Warmup)
 Chain 3 Iteration: 500 / 2000 [25%] (Warmup)
 Chain 4 Iteration: 600 / 2000 [30%] (Warmup)
 Chain 1 Iteration: 600 / 2000 [30%] (Warmup)
 Chain 2 Iteration: 800 / 2000 [40%] (Warmup)
 Chain 3 Iteration: 600 / 2000 [30%] (Warmup)
 Chain 4 Iteration: 700 / 2000 [35%] (Warmup)
 Chain 1 Iteration: 700 / 2000 [35%] (Warmup)
 Chain 2 Iteration: 900 / 2000 [45%] (Warmup)
 Chain 3 Iteration: 700 / 2000 [35%] (Warmup)
 Chain 4 Iteration: 800 / 2000 [40%] (Warmup)
 Chain 1 Iteration: 800 / 2000 [40%] (Warmup)
 Chain 2 Iteration: 1000 / 2000 [50%] (Warmup)
 Chain 2 Iteration: 1001 / 2000 [50%] (Sampling)
 Chain 3 Iteration: 800 / 2000 [40%] (Warmup)
 Chain 4 Iteration: 900 / 2000 [45%] (Warmup)
 Chain 1 Iteration: 900 / 2000 [45%] (Warmup)
 Chain 2 Iteration: 1100 / 2000 [55%] (Sampling)
 Chain 3 Iteration: 900 / 2000 [45%] (Warmup)
 Chain 4 Iteration: 1000 / 2000 [50%] (Warmup)
 Chain 4 Iteration: 1001 / 2000 [50%] (Sampling)
 Chain 1 Iteration: 1000 / 2000 [50%] (Warmup)
 Chain 1 Iteration: 1001 / 2000 [50%] (Sampling)
 Chain 2 Iteration: 1200 / 2000 [60%] (Sampling)
 Chain 4 Iteration: 1100 / 2000 [55%] (Sampling)
 Chain 2 Iteration: 1300 / 2000 [65%] (Sampling)
 Chain 3 Iteration: 1000 / 2000 [50%] (Warmup)
 Chain 3 Iteration: 1001 / 2000 [50%] (Sampling)
 Chain 1 Iteration: 1100 / 2000 [55%] (Sampling)
 Chain 3 Iteration: 1100 / 2000 [55%] (Sampling)
 Chain 4 Iteration: 1200 / 2000 [60%] (Sampling)
 Chain 1 Iteration: 1200 / 2000 [60%] (Sampling)
 Chain 2 Iteration: 1400 / 2000 [70%] (Sampling)
 Chain 4 Iteration: 1300 / 2000 [65%] (Sampling)
 Chain 3 Iteration: 1200 / 2000 [60%] (Sampling)
 Chain 1 Iteration: 1300 / 2000 [65%] (Sampling)
 Chain 2 Iteration: 1500 / 2000 [75%] (Sampling)
 Chain 4 Iteration: 1400 / 2000 [70%] (Sampling)
 Chain 1 Iteration: 1400 / 2000 [70%] (Sampling)
 Chain 2 Iteration: 1600 / 2000 [80%] (Sampling)
 Chain 3 Iteration: 1300 / 2000 [65%] (Sampling)
 Chain 4 Iteration: 1500 / 2000 [75%] (Sampling)
 Chain 1 Iteration: 1500 / 2000 [75%] (Sampling)
 Chain 2 Iteration: 1700 / 2000 [85%] (Sampling)
 Chain 3 Iteration: 1400 / 2000 [70%] (Sampling)

```

Chain 1 Iteration: 1600 / 2000 [ 80%] (Sampling)
Chain 4 Iteration: 1600 / 2000 [ 80%] (Sampling)
Chain 2 Iteration: 1800 / 2000 [ 90%] (Sampling)
Chain 3 Iteration: 1500 / 2000 [ 75%] (Sampling)
Chain 1 Iteration: 1700 / 2000 [ 85%] (Sampling)
Chain 2 Iteration: 1900 / 2000 [ 95%] (Sampling)
Chain 4 Iteration: 1700 / 2000 [ 85%] (Sampling)
Chain 3 Iteration: 1600 / 2000 [ 80%] (Sampling)
Chain 1 Iteration: 1800 / 2000 [ 90%] (Sampling)
Chain 4 Iteration: 1800 / 2000 [ 90%] (Sampling)
Chain 2 Iteration: 2000 / 2000 [100%] (Sampling)
Chain 2 finished in 7.1 seconds.
Chain 3 Iteration: 1700 / 2000 [ 85%] (Sampling)
Chain 1 Iteration: 1900 / 2000 [ 95%] (Sampling)
Chain 4 Iteration: 1900 / 2000 [ 95%] (Sampling)
Chain 1 Iteration: 2000 / 2000 [100%] (Sampling)
Chain 3 Iteration: 1800 / 2000 [ 90%] (Sampling)
Chain 1 finished in 7.5 seconds.
Chain 4 Iteration: 2000 / 2000 [100%] (Sampling)
Chain 4 finished in 7.6 seconds.
Chain 3 Iteration: 1900 / 2000 [ 95%] (Sampling)
Chain 3 Iteration: 2000 / 2000 [100%] (Sampling)
Chain 3 finished in 8.0 seconds.

```

All 4 chains finished successfully.
 Mean chain execution time: 7.6 seconds.
 Total execution time: 8.2 seconds.

```

wf_bb <- fit_4pl(std_bb,
  chains = 4, iter = 2000, cores = 4, seed = 123,
  file = file.path(models_dir, "sim_4pl_betabinomial_v2"))

```

Running MCMC with 4 parallel chains...

```

Chain 1 Iteration:    1 / 2000 [ 0%] (Warmup)
Chain 2 Iteration:    1 / 2000 [ 0%] (Warmup)
Chain 3 Iteration:    1 / 2000 [ 0%] (Warmup)
Chain 4 Iteration:    1 / 2000 [ 0%] (Warmup)
Chain 2 Iteration:  100 / 2000 [ 5%] (Warmup)
Chain 4 Iteration:  100 / 2000 [ 5%] (Warmup)
Chain 1 Iteration:  100 / 2000 [ 5%] (Warmup)
Chain 3 Iteration:  100 / 2000 [ 5%] (Warmup)
Chain 1 Iteration:  200 / 2000 [10%] (Warmup)
Chain 2 Iteration:  200 / 2000 [10%] (Warmup)
Chain 3 Iteration:  200 / 2000 [10%] (Warmup)
Chain 4 Iteration:  200 / 2000 [10%] (Warmup)
Chain 1 Iteration:  300 / 2000 [15%] (Warmup)

```

Chain 3 Iteration: 300 / 2000 [15%] (Warmup)
 Chain 2 Iteration: 300 / 2000 [15%] (Warmup)
 Chain 4 Iteration: 300 / 2000 [15%] (Warmup)
 Chain 1 Iteration: 400 / 2000 [20%] (Warmup)
 Chain 3 Iteration: 400 / 2000 [20%] (Warmup)
 Chain 2 Iteration: 400 / 2000 [20%] (Warmup)
 Chain 4 Iteration: 400 / 2000 [20%] (Warmup)
 Chain 1 Iteration: 500 / 2000 [25%] (Warmup)
 Chain 3 Iteration: 500 / 2000 [25%] (Warmup)
 Chain 2 Iteration: 500 / 2000 [25%] (Warmup)
 Chain 4 Iteration: 500 / 2000 [25%] (Warmup)
 Chain 1 Iteration: 600 / 2000 [30%] (Warmup)
 Chain 3 Iteration: 600 / 2000 [30%] (Warmup)
 Chain 2 Iteration: 600 / 2000 [30%] (Warmup)
 Chain 4 Iteration: 600 / 2000 [30%] (Warmup)
 Chain 1 Iteration: 700 / 2000 [35%] (Warmup)
 Chain 3 Iteration: 700 / 2000 [35%] (Warmup)
 Chain 2 Iteration: 700 / 2000 [35%] (Warmup)
 Chain 4 Iteration: 700 / 2000 [35%] (Warmup)
 Chain 1 Iteration: 800 / 2000 [40%] (Warmup)
 Chain 3 Iteration: 800 / 2000 [40%] (Warmup)
 Chain 2 Iteration: 800 / 2000 [40%] (Warmup)
 Chain 4 Iteration: 800 / 2000 [40%] (Warmup)
 Chain 1 Iteration: 900 / 2000 [45%] (Warmup)
 Chain 3 Iteration: 900 / 2000 [45%] (Warmup)
 Chain 2 Iteration: 900 / 2000 [45%] (Warmup)
 Chain 4 Iteration: 900 / 2000 [45%] (Warmup)
 Chain 1 Iteration: 1000 / 2000 [50%] (Warmup)
 Chain 1 Iteration: 1001 / 2000 [50%] (Sampling)
 Chain 3 Iteration: 1000 / 2000 [50%] (Warmup)
 Chain 3 Iteration: 1001 / 2000 [50%] (Sampling)
 Chain 2 Iteration: 1000 / 2000 [50%] (Warmup)
 Chain 2 Iteration: 1001 / 2000 [50%] (Sampling)
 Chain 4 Iteration: 1000 / 2000 [50%] (Warmup)
 Chain 4 Iteration: 1001 / 2000 [50%] (Sampling)
 Chain 1 Iteration: 1100 / 2000 [55%] (Sampling)
 Chain 3 Iteration: 1100 / 2000 [55%] (Sampling)
 Chain 2 Iteration: 1100 / 2000 [55%] (Sampling)
 Chain 4 Iteration: 1100 / 2000 [55%] (Sampling)
 Chain 1 Iteration: 1200 / 2000 [60%] (Sampling)
 Chain 3 Iteration: 1200 / 2000 [60%] (Sampling)
 Chain 2 Iteration: 1200 / 2000 [60%] (Sampling)
 Chain 4 Iteration: 1200 / 2000 [60%] (Sampling)
 Chain 1 Iteration: 1300 / 2000 [65%] (Sampling)
 Chain 3 Iteration: 1300 / 2000 [65%] (Sampling)
 Chain 4 Iteration: 1300 / 2000 [65%] (Sampling)
 Chain 1 Iteration: 1400 / 2000 [70%] (Sampling)
 Chain 2 Iteration: 1300 / 2000 [65%] (Sampling)

```

Chain 3 Iteration: 1400 / 2000 [ 70%] (Sampling)
Chain 4 Iteration: 1400 / 2000 [ 70%] (Sampling)
Chain 1 Iteration: 1500 / 2000 [ 75%] (Sampling)
Chain 2 Iteration: 1400 / 2000 [ 70%] (Sampling)
Chain 3 Iteration: 1500 / 2000 [ 75%] (Sampling)
Chain 4 Iteration: 1500 / 2000 [ 75%] (Sampling)
Chain 1 Iteration: 1600 / 2000 [ 80%] (Sampling)
Chain 3 Iteration: 1600 / 2000 [ 80%] (Sampling)
Chain 2 Iteration: 1500 / 2000 [ 75%] (Sampling)
Chain 4 Iteration: 1600 / 2000 [ 80%] (Sampling)
Chain 1 Iteration: 1700 / 2000 [ 85%] (Sampling)
Chain 3 Iteration: 1700 / 2000 [ 85%] (Sampling)
Chain 2 Iteration: 1600 / 2000 [ 80%] (Sampling)
Chain 4 Iteration: 1700 / 2000 [ 85%] (Sampling)
Chain 1 Iteration: 1800 / 2000 [ 90%] (Sampling)
Chain 3 Iteration: 1800 / 2000 [ 90%] (Sampling)
Chain 2 Iteration: 1700 / 2000 [ 85%] (Sampling)
Chain 4 Iteration: 1800 / 2000 [ 90%] (Sampling)
Chain 1 Iteration: 1900 / 2000 [ 95%] (Sampling)
Chain 3 Iteration: 1900 / 2000 [ 95%] (Sampling)
Chain 4 Iteration: 1900 / 2000 [ 95%] (Sampling)
Chain 2 Iteration: 1800 / 2000 [ 90%] (Sampling)
Chain 1 Iteration: 2000 / 2000 [100%] (Sampling)
Chain 3 Iteration: 2000 / 2000 [100%] (Sampling)
Chain 1 finished in 10.8 seconds.
Chain 3 finished in 10.8 seconds.
Chain 4 Iteration: 2000 / 2000 [100%] (Sampling)
Chain 4 finished in 11.0 seconds.
Chain 2 Iteration: 1900 / 2000 [ 95%] (Sampling)
Chain 2 Iteration: 2000 / 2000 [100%] (Sampling)
Chain 2 finished in 11.6 seconds.

```

All 4 chains finished successfully.
 Mean chain execution time: 11.1 seconds.
 Total execution time: 11.7 seconds.

```

# Convenience: extract the brmsfit object for downstream brms helpers.
fit_bin <- wf_bin$fit
fit_bb  <- wf_bb$fit

```

We can also get an idea of how much variation our model explains that considers all sources of relevant variation.

```

# Obtain the Bayes R2 for each model
brms::bayes_R2(fit_bin)

```

Estimate	Est.Error	Q2.5	Q97.5
----------	-----------	------	-------

Table S3: Posterior medians and 95% credible intervals for the four 4PL parameters and the temperature slope, compared to the known truth used to generate the simulated data. Recovery is judged successful when the credible interval covers the truth.

Parameter	Truth	Bin. median	Bin. lower	Bin. upper	BB median	BB lower	BB upper
ell	0.03	0.0313	0.0268	0.036	0.0302	0.0199	0.0409
u	0.97	0.9707	0.9669	0.974	0.9707	0.9624	0.9781
k	8	8.0948	7.704	8.505	7.7165	6.8915	8.684
mid (intercept)	1.5	1.4976	1.4891	1.507	1.5054	1.4847	1.5258
mid (slope)	-0.15	-0.1497	-0.1527	-0.147	-0.1544	-0.1617	-0.1471

R2 0.9885111 0.000117092 0.9882313 0.9886848

```
brms::bayes_R2(fit_bb)
```

	Estimate	Est.Error	Q2.5	Q97.5
R2	0.9278098	0.001148178	0.9251855	0.9296789

2.0.3.3 Checking parameter recovery

We can now check how well our fitted model recovered the true parameters that generated the data. The most direct check is to compare posterior medians (with 95% credible intervals) to the known truth (Table S3). We have big sample sizes here so our values should be pretty close to our true values.

We can also overlay the posterior fitted curves on the simulated data (Figure S3). If the model has recovered the true surface, the fitted lines should track the simulated proportions at every assay temperature.

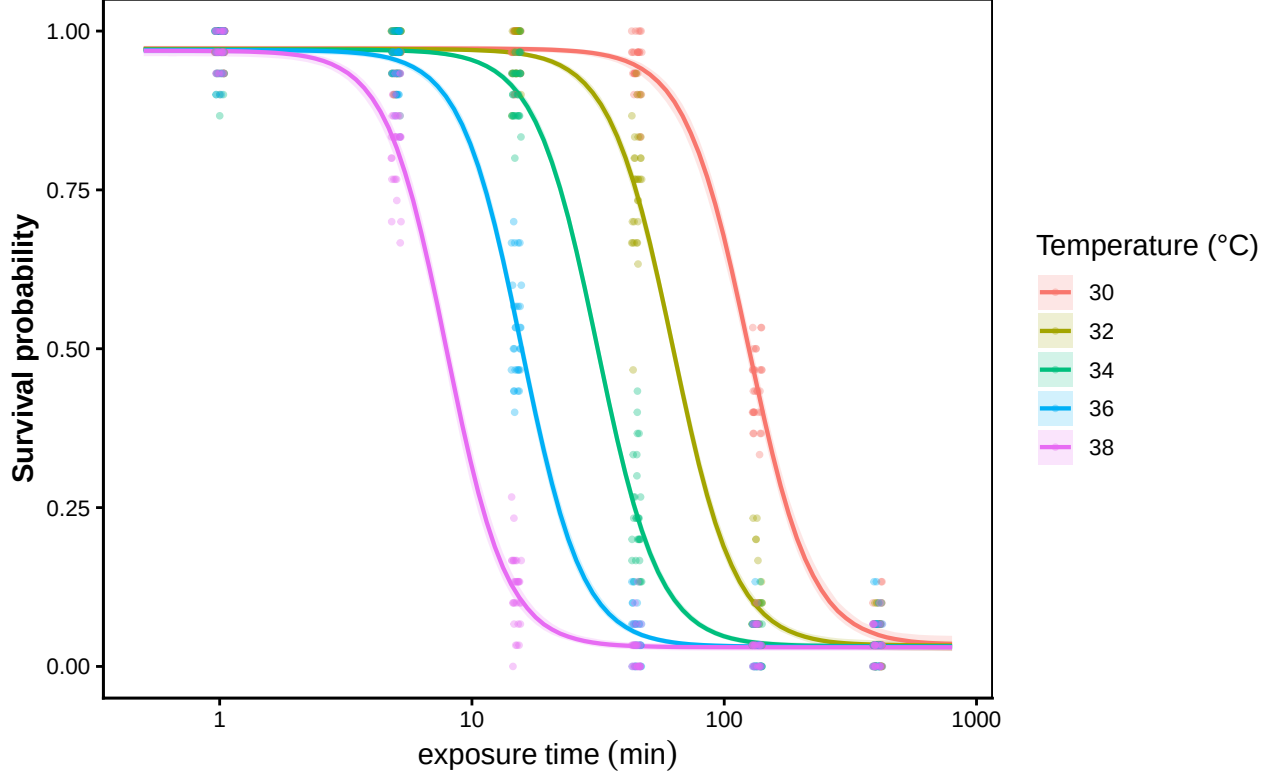


Figure S3: Posterior fitted 4PL curves (solid lines, with shaded 95% credible bands) overlaid on the simulated binomial data (points). One panel-coloured line per assay temperature. Where the fitted curves track the simulated proportions across all five temperatures, the model has recovered the underlying dose-response surface.

2.0.4 Deriving z , $CT_{max_{1hr}}$, and T_{crit} from the posterior

The whole point of fitting the joint Bayesian 4PL is that the classical TLS quantities fall out as transformations of the posterior we just fit — no second regression, no bootstrap. `extract_tdt()` bundles three of them:

- z — thermal sensitivity, derived by a per-draw log-linear regression of $\log_{10}$ LT50 on temperature (Equation 7 of the manuscript).
- $CT_{max_{1hr}}$ — temperature at 50% survival after 1 hour exposure, obtained by inverting the fitted 4PL at the reference time, per posterior draw.
- T_{crit} — the temperature below which damage accumulates more slowly than a chosen rate floor. The damage rate at temperature T in the TDT framework is $r(T) = 100 \cdot 10^{(T-CT_{max,1hr})/z}$ % LT50-dose per hour, so inverting for a chosen rate floor r^* gives

$$T_{crit}(r^*) = CT_{max,1hr} + z \cdot \log_{10}(r^*/100). \quad (S3)$$

Different r^* values give different T_{crit} thresholds, and the literature does not converge on a single value. Empirical breakpoint analyses across taxa as different as *Drosophila suzukii* and *Lemna gibba*

fall in the range $r^* \in [0.1, 1]$ % HI per hour, corresponding to “ ~ 4 days to 1000 hours of continuous exposure to reach LT_{50} ” [Jørgensen *et al.* (2021); Arnold *et al.* (2025); **havro_faber_2023**]. Rather than commit to one r^* , we treat it as an unknown with a uniform prior on $\log_{10} r^*$ over that range and integrate; the posterior of T_{crit} then carries both parameter uncertainty (in $CT_{max,1hr}$ and z) and operational uncertainty (in the choice of r^*). The median of that pooled posterior sits at $CT_{max,1hr} - 2.5 \cdot z$, which corresponds to $r^* \approx 0.3$ % HI per hour (the geometric mean of the rate range).

All three quantities inherit the same posterior, so every credible interval reflects joint uncertainty in $(\ell, u, k, \beta_0, \beta_1)$.

i Why ℓ , u and k varying with T does not break this recipe

The default `fit_4pl()` puts a `temp_c` slope on every 4PL sub-parameter, so ℓ , u and k can each carry a linear effect of temperature in addition to the linear-mid term — when those slopes shrink to zero under the prior the model reduces cleanly to the constant-shape case. When the data carry real temperature signal on any of the shape parameters, the asymmetry correction $\frac{1}{k(T)} \log \frac{u(T)-0.5}{0.5-\ell(T)}$ varies with T and the resulting $\log_{10} LT50(T)$ curve is bent rather than straight:

$$\log_{10} LT50(T) = \beta_0 + \beta_1(T - \bar{T}) + \frac{1}{k(T)} \log \frac{u(T)-0.5}{0.5-\ell(T)}. \quad (S4)$$

`extract_tdt()` requires no special handling in either regime, because both of its primitives operate on the *fitted 4PL surface* rather than on a closed-form expression: $CT_{max,1hr}$ is obtained by inverting the surface at the reference duration t_{ref} per posterior draw (`derive_temperature_for_duration()`), and the per-draw OLS regression that yields z runs over the per-draw $\log_{10} LT50(T)$ curve recovered numerically by `derive_ltx_curve()`. When the curve is straight, the OLS slope returns the closed-form z ; when it is bent, the same OLS returns the *average local slope* across the assay grid — a sensible single-summary z , with the per-temperature local values accessible from the underlying $LT50(T)$ curve when needed. The worked example in Section 2.0.6 demonstrates this explicitly with a simulation where u and k carry strong T effects.

```
# Data are in minutes (standardize_data was called with duration_unit = "minutes"),
# so set time_multiplier = 1 to keep the model's time axis unchanged on output,
# and t_ref = 60 to read CTmax at the 1-hour reference. T_crit uses the
# rate-multiplier integration; default TC_rate_range = c(0.1, 1) % HI per hour.
et_bin <- extract_tdt(wf_bin, t_ref = 60, time_multiplier = 1, ndraws = 1000)
et_bb  <- extract_tdt(wf_bb, t_ref = 60, time_multiplier = 1, ndraws = 1000)
```

Table S4: Posterior medians and 95% credible intervals for thermal sensitivity (z), the critical thermal limit at a 1-hour reference duration ($CT_{max_{1hr}}$), and the rate-multiplier critical temperature (T_{crit} , integrated over $r^* \in [0.1, 1]$ % HI per hour). All three derived via `extract_tdt()` from a single posterior. Each row's truth value is computed analytically from `true_params`.

Quantity	Truth	Bin. median	Bin. lower	Bin. upper	BB median	BB lower	BB upper
z (°C)	6.67	6.67	6.55	6.8	6.49	6.22	6.8
CTmax at 1 hr (°C)	32.15	32.13	32.07	32.2	32.23	32.08	32.4
T_{crit} (°C)	15.48	15.45	12.23	18.6	15.96	12.72	19.2

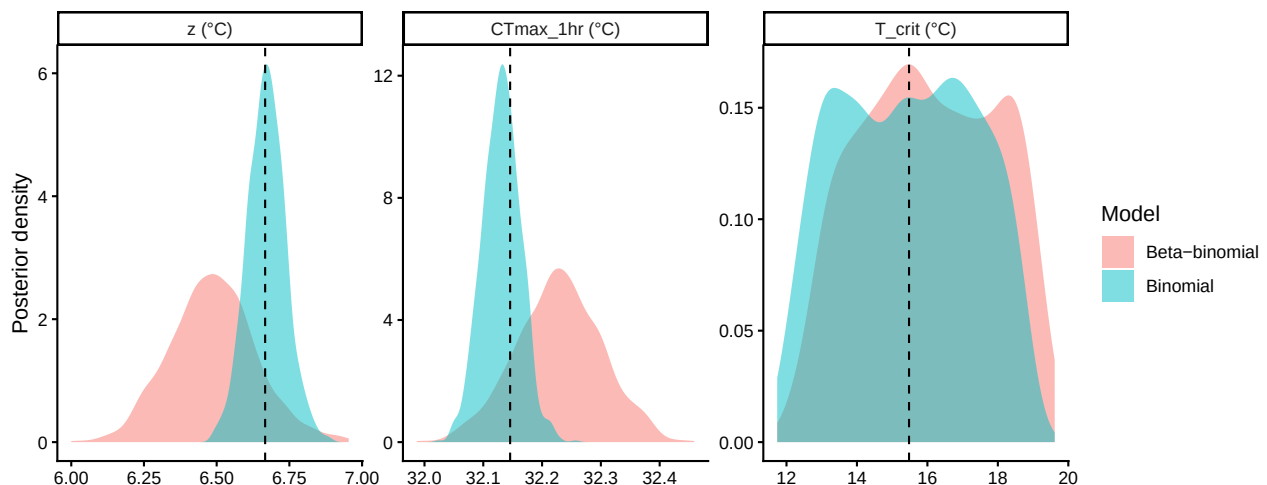


Figure S4: Posterior densities of z , $CT_{max_{1hr}}$, and T_{crit} from the binomial (blue) and beta-binomial (orange) joint fits. Vertical dashed lines mark the simulation truth. All three posteriors concentrate around their respective truths — illustrating that one fit, one library call (`extract_tdt()`), produces every classical TLS quantity with calibrated uncertainty and no extra regression step.

All three quantities recover their simulation truth in both fits (Table S4, Figure S4), with credible intervals reflecting joint uncertainty in all 4PL parameters. The key practical advantage of the joint approach over the two-step pipeline: every classical TLS quantity is one library call away from the fit we already have, and every quantity inherits the same calibrated uncertainty.

Posterior summary statistics — mean, SD, median, and 95% credible interval — are also readily available from the `$draws` slot of `extract_tdt()`'s output:

```
summarise_draws <- function(x) {
  q <- stats::quantile(x, c(0.025, 0.5, 0.975), names = FALSE, na.rm = TRUE)
  tibble::tibble(mean = mean(x, na.rm = TRUE),
                 sd   = stats::sd(x, na.rm = TRUE),
                 median = q[2], lower = q[1], upper = q[3])
}
```

Table S5: Posterior summaries of z , $CT_{max_{1hr}}$, and T_{crit} from the joint Bayesian 4PL: posterior mean, standard deviation, median, and 95% credible interval, reported separately for the binomial and beta-binomial fits.

Quantity	Likelihood	Mean	SD	Median	Lower	Upper
z (°C)	Binomial	6.67	0.0652	6.67	6.55	6.8
CTmax at 1 hr (°C)	Binomial	32.13	0.0322	32.13	32.07	32.2
T_{crit} (°C)	Binomial	15.44	1.8965	15.45	12.23	18.6
z (°C)	Beta-binomial	6.49	0.1478	6.49	6.22	6.8
CTmax at 1 hr (°C)	Beta-binomial	32.23	0.0723	32.23	32.08	32.4
T_{crit} (°C)	Beta-binomial	16.02	1.9075	15.96	12.72	19.2

```

uncert_summary <- dplyr::bind_rows(
  summarise_draws(et_bin$z$draws$z)      |> dplyr::mutate(quantity = "z (°C)",
  summarise_draws(et_bin$CTmax$draws$temp) |> dplyr::mutate(quantity = "CTmax at 1 hr (°C)",
  summarise_draws(et_bin$T_crit$draws$temp) |> dplyr::mutate(quantity = "T_crit (°C)", likelihood = "Binomial",
  summarise_draws(et_bb$z$draws$z)      |> dplyr::mutate(quantity = "z (°C)",
  summarise_draws(et_bb$CTmax$draws$temp) |> dplyr::mutate(quantity = "CTmax at 1 hr (°C)",
  summarise_draws(et_bb$T_crit$draws$temp) |> dplyr::mutate(quantity = "T_crit (°C)", likelihood = "Beta-binomial")
) |>
  dplyr::select(quantity, likelihood, mean, sd, median, lower, upper)

# Scalars for inline reporting in the next paragraph.
sd_z_bin   <- round(stats::sd(et_bin$z$draws$z),      2)
sd_z_bb    <- round(stats::sd(et_bb$z$draws$z),      2)
sd_ct_bin  <- round(stats::sd(et_bin$CTmax$draws$temp), 2)
sd_ct_bb   <- round(stats::sd(et_bb$CTmax$draws$temp), 2)
ci_z_bin   <- round(diff(stats::quantile(et_bin$z$draws$z,      c(0.025, 0.975), names = FALSE),
ci_z_bb    <- round(diff(stats::quantile(et_bb$z$draws$z,      c(0.025, 0.975), names = FALSE),
ci_ct_bin  <- round(diff(stats::quantile(et_bin$CTmax$draws$temp, c(0.025, 0.975), names = FALSE),
ci_ct_bb   <- round(diff(stats::quantile(et_bb$CTmax$draws$temp, c(0.025, 0.975), names = FALSE),

uncert_summary |>
  dplyr::rename(
    Quantity   = quantity,
    Likelihood = likelihood,
    Mean       = mean,
    SD         = sd,
    Median     = median,
    Lower      = lower,
    Upper      = upper
  ) |>
  tinytable::tt(digits = 3, escape = TRUE)

```

In short, both z and $CT_{max_{1hr}}$ are estimated with high precision and their credible intervals come

Table S6: Point estimates (two-stage) and posterior medians (joint Bayesian) for thermal sensitivity (z) and $CT_{max_{1hr}}$, recovered by each of four estimator–dataset combinations from the same simulated datasets. Truth values are the simulation parameters.

Quantity	Truth	Two-stage (binomial)	Joint Bayesian (binomial)	Two-stage (beta-binomial)
z (°C)	6.67	6.91	6.67	6.9
CTmax at 1 hr (°C)	32.15	32.03	32.13	32.1

straight from the joint posterior. If we fit this model using frequentist approaches we could do the same and use the delta-method, or bootstrapping to get uncertainty.

Under the binomial likelihood, the standard error (or SD of posterior) for z is 0.07 °C with a 95% credible interval 0.26 °C wide; $CT_{max_{1hr}}$ has SD 0.03 °C and a 95% credible interval width of 0.12 °C.

The beta-binomial fit is slightly wider on both quantities (SD on z : 0.15 °C; on $CT_{max_{1hr}}$: 0.07 °C) because ϕ has absorbed the extra-binomial variance and that uncertainty propagates honestly into the derived quantities — none of which falls out of the two-stage pipeline without an additional bootstrap or delta-method step.

! Important

It is always important to evaluate whether a binomial and a beta-binomial model better fits real data. The extra dispersion, typical of real biological data, should be estimated when it's present as it has important consequences for inferences.

2.0.5 Comparing the two pipelines

2.0.5.1 Point-estimate agreement on the same data

Table S6 puts the two-stage point estimates next to the joint Bayesian posterior medians for both simulated datasets.

All four estimators land on essentially the same point estimates for z and $CT_{max_{1hr}}$, though we notice the two stage is biased upwards slightly, and all sit close to the simulation truth. The joint Bayesian 4PL gives the same answer the classical pipeline gives — it is *not* in disagreement with the field's existing point estimates, it reproduces them.

2.0.6 A worked example: temperature effects on every shape parameter

The default `fit_4pl()` already puts a `temp_c` slope on ℓ , u and k , so any temperature effect on the shape parameters is fit alongside the linear midpoint. To validate that `extract_tdt()` returns sensible z , $CT_{max_{1hr}}$ and $\log_{10} \text{LT50}(T)$ posteriors when the truth violates the constant-shape assumption, we re-simulate beta-binomial data on the same factorial design as Section 2.0.1.2, but now u declines steeply with T and k rises steeply with T . The fit is then exactly the same `fit_4pl()` call as before — no special formula, no special priors — and the analysis afterwards is exactly the same `extract_tdt()` call.

```

# Same factorial design as @sec-sim-step2; re-declared here so this chunk is
# self-contained.
temps      <- c(30, 32, 34, 36, 38)
durations  <- c(1, 5, 15, 45, 135, 405)
n_rep      <- 30
n_per_rep  <- 30
phi        <- 5

# Truth on the same raw scale fit_4pl() uses internally (so we can compare the
# posterior coefficients to truth directly):
# ell = 0.49 * inv_logit(ell_raw)                (held constant in T)
# u   = 0.51 + 0.49 * inv_logit(u_raw_0 + u_raw_T*T_c) (declines in T)
# k   = exp(log_k_0 + log_k_T*T_c)                (rises in T)
true_params_ext <- list(
  ell_raw = qlogis(0.05 / 0.49),                # ell = 0.05
  u_raw_0 = qlogis((0.92 - 0.51) / 0.49),        # u = 0.92 at T_bar
  u_raw_T = -0.60,                               # strong decline in u with T
  log_k_0 = log(8),                               # k = 8 at T_bar
  log_k_T = 0.30,                                 # strong rise in k with T
  m_beta0 = 1.5,
  m_beta1 = -0.15,
  T_bar   = 34
)

design <- tidyr::expand_grid(
  T = temps,
  t = durations,
  rep = seq_len(n_rep)
) |>
dplyr::mutate(
  log10_t = log10(t),
  T_c     = T - true_params_ext$T_bar,
  n       = n_per_rep
)

sim_bb_ext <- design |>
dplyr::mutate(
  ell = 0.49 * plogis(true_params_ext$ell_raw),
  u   = 0.51 + 0.49 * plogis(true_params_ext$u_raw_0 +
                             true_params_ext$u_raw_T * T_c),
  k   = exp(true_params_ext$log_k_0 + true_params_ext$log_k_T * T_c),
  mid = true_params_ext$m_beta0 + true_params_ext$m_beta1 * T_c,
  p_true = ell + (u - ell) / (1 + exp(k * (log10_t - mid))),
  p_draw = rbeta(dplyr::n(), p_true * phi, (1 - p_true) * phi),
  y      = rbinom(dplyr::n(), size = n, prob = p_draw)
)

```

Under this truth the lower asymptote stays flat at $\ell = 0.05$, the upper asymptote u falls from 0.99 at $T = 30$ to 0.67 at $T = 38$, and the steepness k rises from 2.4 to 26.6 across the same range. The resulting dose-response curves are visibly different in shape at the two extremes (Figure S5).

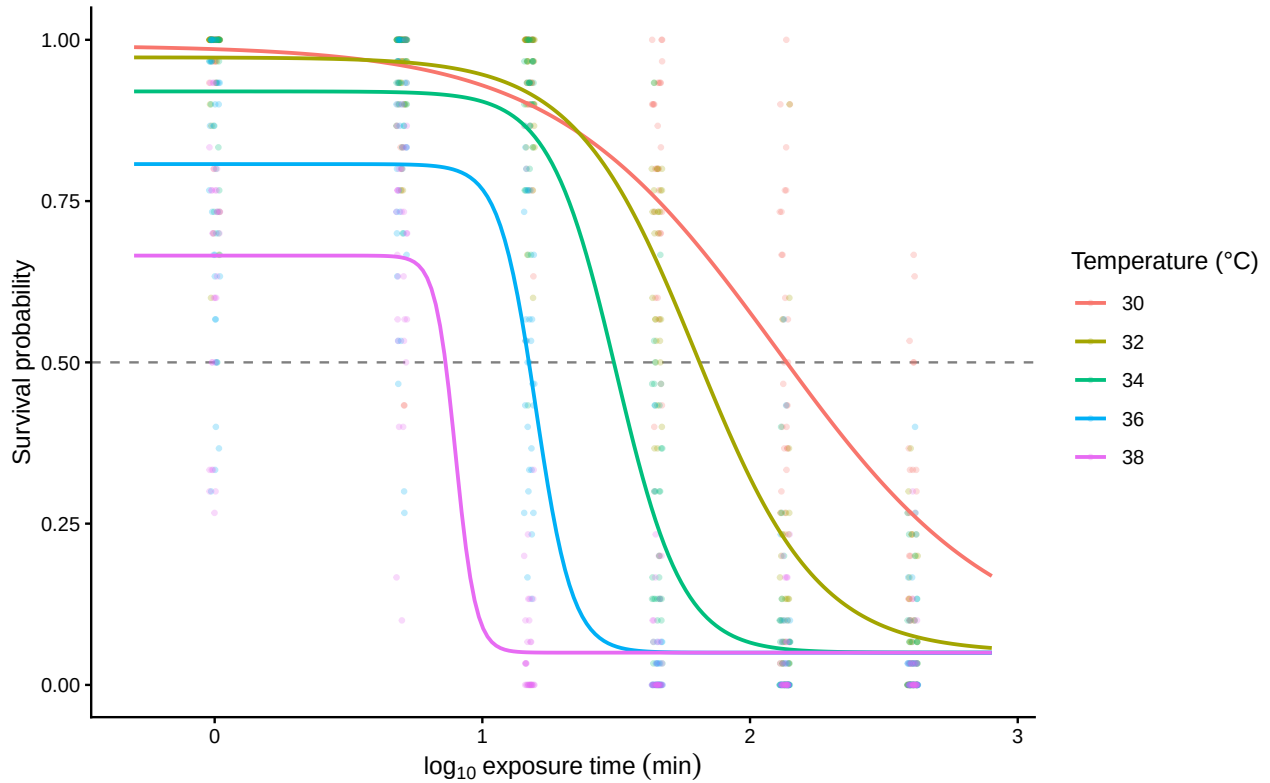


Figure S5: True 4PL dose-response surface (solid lines) for the extended simulation, where k rises and u falls with assay temperature, overlaid on the simulated beta-binomial counts (points). Note how the curves at $T = 30$ and $T = 38$ have very different upper asymptotes and steepness — not just a horizontal shift along the time axis.

Before fitting we can compute, analytically, what the truth implies for $\log_{10} \text{LT50}(T)$, the local thermal sensitivity $z(T)$ at each assay temperature (defined as -1 divided by the local slope of $\log_{10} \text{LT50}$ on T), and $CT_{\max_{1hr}}$ (the T at which $\log_{10} \text{LT50}$ crosses $\log_{10} 60$). These are the targets we will ask `extract_tdt()` to recover.

```
# True log10(LT50) at any T from the T-varying 4PL: solve p(log10_t) = 0.5
# for log10_t. With the 4PL,
# log10_t = mid + (1/k) * log((u - 0.5) / (0.5 - ell))
true_log10_LT50 <- function(T_value) {
  T_c <- T_value - true_params_ext$T_bar
  ell <- 0.49 * plogis(true_params_ext$ell_raw)
  u <- 0.51 + 0.49 * plogis(true_params_ext$u_raw_0 +
                           true_params_ext$u_raw_T * T_c)
  k <- exp(true_params_ext$log_k_0 + true_params_ext$log_k_T * T_c)
  mid <- true_params_ext$m_beta0 + true_params_ext$m_beta1 * T_c
  mid + (1 / k) * log((u - 0.5) / (0.5 - ell))
}
```

```

}

# Local z(T) via central finite difference of true log10(LT50)(T).
true_local_z <- function(T_at, half_step = 0.5) {
  -1 / ((true_log10_LT50(T_at + half_step) -
        true_log10_LT50(T_at - half_step)) / (2 * half_step))
}

# True CTmax_1hr: search the fine grid for the T at which log10(LT50)
# crosses log10(60).
T_grid_truth <- seq(20, 45, by = 0.001)
ll_grid_truth <- vapply(T_grid_truth, true_log10_LT50, numeric(1))
true_CTmax_1hr <- T_grid_truth[which.min(abs(ll_grid_truth - log10(60)))]

truth_ext <- tibble::tibble(
  T = temps,
  log10_LT50_true = vapply(temps, true_log10_LT50, numeric(1)),
  LT50_min_true = 10 ^ log10_LT50_true,
  local_z_true = vapply(temps, true_local_z, numeric(1))
)

true_mean_local_z <- mean(truth_ext$local_z_true)

```

Across the assay range the true $\log_{10} \text{LT50}(T)$ curve is bent: the implied local z varies from 6.05 °C at $T = 30$ to 6.53 °C at $T = 38$ (truth mean $z = 6.28$ °C). The truth implies $CT_{max_{1hr}} = 32.21$ °C. With straight-line $\log_{10} \text{LT50}(T)$ — the constant-shape case — local z would not vary across temperature at all.

We now fit with exactly the same `fit_4pl()` call as before. The default formula already puts a `temp_c` slope on each of the four 4PL sub-parameters, so the model has the capacity to recover the truth's temperature signal on u and k without any modification.

```

std_ext <- standardize_data(sim_bb_ext,
  temp = "T", duration = "t",
  n_total = "n", n_surv = "y",
  duration_unit = "minutes")

wf_ext <- fit_4pl(std_ext,
  chains = 4, iter = 2000, cores = 4, seed = 123,
  file = file.path(models_dir,
    "sim_4pl_ext_betabin_DEFAULT_strong_v1"))

```

Running MCMC with 4 parallel chains...

```

Chain 1 Iteration:    1 / 2000 [  0%] (Warmup)
Chain 2 Iteration:    1 / 2000 [  0%] (Warmup)
Chain 3 Iteration:    1 / 2000 [  0%] (Warmup)

```


Chain 4 Iteration: 1 / 2000 [0%] (Warmup)
 Chain 1 Iteration: 100 / 2000 [5%] (Warmup)
 Chain 4 Iteration: 100 / 2000 [5%] (Warmup)
 Chain 3 Iteration: 100 / 2000 [5%] (Warmup)
 Chain 2 Iteration: 100 / 2000 [5%] (Warmup)
 Chain 1 Iteration: 200 / 2000 [10%] (Warmup)
 Chain 4 Iteration: 200 / 2000 [10%] (Warmup)
 Chain 3 Iteration: 200 / 2000 [10%] (Warmup)
 Chain 2 Iteration: 200 / 2000 [10%] (Warmup)
 Chain 1 Iteration: 300 / 2000 [15%] (Warmup)
 Chain 4 Iteration: 300 / 2000 [15%] (Warmup)
 Chain 3 Iteration: 300 / 2000 [15%] (Warmup)
 Chain 2 Iteration: 300 / 2000 [15%] (Warmup)
 Chain 1 Iteration: 400 / 2000 [20%] (Warmup)
 Chain 4 Iteration: 400 / 2000 [20%] (Warmup)
 Chain 2 Iteration: 400 / 2000 [20%] (Warmup)
 Chain 3 Iteration: 400 / 2000 [20%] (Warmup)
 Chain 1 Iteration: 500 / 2000 [25%] (Warmup)
 Chain 4 Iteration: 500 / 2000 [25%] (Warmup)
 Chain 2 Iteration: 500 / 2000 [25%] (Warmup)
 Chain 1 Iteration: 600 / 2000 [30%] (Warmup)
 Chain 3 Iteration: 500 / 2000 [25%] (Warmup)
 Chain 4 Iteration: 600 / 2000 [30%] (Warmup)
 Chain 2 Iteration: 600 / 2000 [30%] (Warmup)
 Chain 1 Iteration: 700 / 2000 [35%] (Warmup)
 Chain 3 Iteration: 600 / 2000 [30%] (Warmup)
 Chain 4 Iteration: 700 / 2000 [35%] (Warmup)
 Chain 2 Iteration: 700 / 2000 [35%] (Warmup)
 Chain 1 Iteration: 800 / 2000 [40%] (Warmup)
 Chain 3 Iteration: 700 / 2000 [35%] (Warmup)
 Chain 4 Iteration: 800 / 2000 [40%] (Warmup)
 Chain 2 Iteration: 800 / 2000 [40%] (Warmup)
 Chain 1 Iteration: 900 / 2000 [45%] (Warmup)
 Chain 4 Iteration: 900 / 2000 [45%] (Warmup)
 Chain 3 Iteration: 800 / 2000 [40%] (Warmup)
 Chain 2 Iteration: 900 / 2000 [45%] (Warmup)
 Chain 3 Iteration: 900 / 2000 [45%] (Warmup)
 Chain 1 Iteration: 1000 / 2000 [50%] (Warmup)
 Chain 1 Iteration: 1001 / 2000 [50%] (Sampling)
 Chain 4 Iteration: 1000 / 2000 [50%] (Warmup)
 Chain 4 Iteration: 1001 / 2000 [50%] (Sampling)
 Chain 2 Iteration: 1000 / 2000 [50%] (Warmup)
 Chain 2 Iteration: 1001 / 2000 [50%] (Sampling)
 Chain 3 Iteration: 1000 / 2000 [50%] (Warmup)
 Chain 3 Iteration: 1001 / 2000 [50%] (Sampling)
 Chain 4 Iteration: 1100 / 2000 [55%] (Sampling)
 Chain 1 Iteration: 1100 / 2000 [55%] (Sampling)
 Chain 2 Iteration: 1100 / 2000 [55%] (Sampling)

```

Chain 4 Iteration: 1200 / 2000 [ 60%] (Sampling)
Chain 3 Iteration: 1100 / 2000 [ 55%] (Sampling)
Chain 1 Iteration: 1200 / 2000 [ 60%] (Sampling)
Chain 2 Iteration: 1200 / 2000 [ 60%] (Sampling)
Chain 4 Iteration: 1300 / 2000 [ 65%] (Sampling)
Chain 3 Iteration: 1200 / 2000 [ 60%] (Sampling)
Chain 1 Iteration: 1300 / 2000 [ 65%] (Sampling)
Chain 2 Iteration: 1300 / 2000 [ 65%] (Sampling)
Chain 4 Iteration: 1400 / 2000 [ 70%] (Sampling)
Chain 3 Iteration: 1300 / 2000 [ 65%] (Sampling)
Chain 1 Iteration: 1400 / 2000 [ 70%] (Sampling)
Chain 2 Iteration: 1400 / 2000 [ 70%] (Sampling)
Chain 4 Iteration: 1500 / 2000 [ 75%] (Sampling)
Chain 3 Iteration: 1400 / 2000 [ 70%] (Sampling)
Chain 4 Iteration: 1600 / 2000 [ 80%] (Sampling)
Chain 1 Iteration: 1500 / 2000 [ 75%] (Sampling)
Chain 2 Iteration: 1500 / 2000 [ 75%] (Sampling)
Chain 4 Iteration: 1700 / 2000 [ 85%] (Sampling)
Chain 3 Iteration: 1500 / 2000 [ 75%] (Sampling)
Chain 1 Iteration: 1600 / 2000 [ 80%] (Sampling)
Chain 2 Iteration: 1600 / 2000 [ 80%] (Sampling)
Chain 4 Iteration: 1800 / 2000 [ 90%] (Sampling)
Chain 3 Iteration: 1600 / 2000 [ 80%] (Sampling)
Chain 1 Iteration: 1700 / 2000 [ 85%] (Sampling)
Chain 2 Iteration: 1700 / 2000 [ 85%] (Sampling)
Chain 4 Iteration: 1900 / 2000 [ 95%] (Sampling)
Chain 3 Iteration: 1700 / 2000 [ 85%] (Sampling)
Chain 2 Iteration: 1800 / 2000 [ 90%] (Sampling)
Chain 4 Iteration: 2000 / 2000 [100%] (Sampling)
Chain 4 finished in 11.1 seconds.
Chain 1 Iteration: 1800 / 2000 [ 90%] (Sampling)
Chain 3 Iteration: 1800 / 2000 [ 90%] (Sampling)
Chain 2 Iteration: 1900 / 2000 [ 95%] (Sampling)
Chain 1 Iteration: 1900 / 2000 [ 95%] (Sampling)
Chain 3 Iteration: 1900 / 2000 [ 95%] (Sampling)
Chain 2 Iteration: 2000 / 2000 [100%] (Sampling)
Chain 2 finished in 12.2 seconds.
Chain 1 Iteration: 2000 / 2000 [100%] (Sampling)
Chain 1 finished in 12.3 seconds.
Chain 3 Iteration: 2000 / 2000 [100%] (Sampling)
Chain 3 finished in 12.5 seconds.

```

All 4 chains finished successfully.

Mean chain execution time: 12.0 seconds.

Total execution time: 12.7 seconds.

The four 4PL sub-parameters on the natural (biological) scale at each assay temperature are recovered in Table S7. The lower asymptote ℓ stays near 0.05 at every T (consistent with the truth that

Table S7: Posterior recovery of the four 4PL sub-parameters on the natural scale at each assay temperature. The fit uses the disjoint-bounds reparameterisation (ℓ , u via `inv_logit`, k via `exp`); raw posterior columns are transformed back to the biological scale before summarising. Posterior medians and 95% credible intervals are reported alongside the analytical truth.

Parameter	T (°C)	Truth	Median	Lower	Upper
ell	30	0.05	0.0527	0.0253	0.0924
u	30	0.991	0.9891	0.9815	0.9943
k	30	2.41	2.5253	2.189	2.9737
mid	30	2.1	2.0704	2.0033	2.1377
ell	32	0.05	0.0511	0.0303	0.0761
u	32	0.973	0.9689	0.9552	0.9805
k	32	4.39	4.3403	3.8439	4.9175
mid	32	1.8	1.7854	1.7425	1.8289
ell	34	0.05	0.0496	0.0355	0.0642
u	34	0.92	0.9141	0.8945	0.933
k	34	8	7.4326	6.1928	9.1234
mid	34	1.5	1.5005	1.473	1.5287
ell	36	0.05	0.0481	0.0386	0.058
u	36	0.807	0.8016	0.7726	0.8271
k	36	14.577	12.7282	9.671	17.5763
mid	36	1.2	1.215	1.1835	1.2517
ell	38	0.05	0.0467	0.035	0.0604
u	38	0.666	0.6615	0.6212	0.7023
k	38	26.561	21.8183	14.8676	34.1307
mid	38	0.9	0.9299	0.8792	0.9892

ℓ does not vary with temperature), while u falls and k rises across the assay range — exactly the truth.

Now we apply the same `extract_tdt()` call as in Section 2.0.4 — there is no separate “T-varying” workflow because `extract_tdt()` already inverts the fitted surface numerically per draw:

```
et_ext <- extract_tdt(wf_ext, t_ref = 60, time_multiplier = 1, ndraws = 1000)
```

Table S8 compares the three classical TDT quantities from `extract_tdt()` to the analytical truth derived above.

Both posteriors comfortably cover their analytical truth. The single-summary z is the OLS slope across the assay grid — which, on a bent curve, estimates the *average* local slope (truth = 6.28 °C, posterior median 6.53 °C, 95% CrI [6.08, 7.04] °C). $CT_{max_{1hr}}$ is recovered by direct 4PL inversion

Table S8: Classical TDT quantities from `extract_tdt()` on the T-varying fit, compared against analytical truth. The single-summary z is the slope of a per-draw OLS regression of $\log_{10} \text{LT50}$ on T across the assay grid; the truth row reports the mean of the analytical local $z(T)$ at the same assay temperatures, which is what the OLS slope estimates when the curve is bent. $CT_{max_{1hr}}$ comes from direct numerical inversion of the fitted 4PL at 60 min per draw.

Quantity	Truth	Median	Lower	Upper
z ($^{\circ}\text{C}$)	6.28	6.53	6.08	7.04
CTmax at 1 hr ($^{\circ}\text{C}$)	32.21	32.12	31.87	32.34
T_crit ($^{\circ}\text{C}$)	NA	15.85	12.22	19.07

at the reference duration and lands on the truth ($32.21^{\circ}\text{C} \rightarrow$ posterior median 32.12°C , 95% CrI $[31.87, 32.34]^{\circ}\text{C}$); the numerical inversion does not depend on the asymmetry correction being constant in T .

The bent $\log_{10} \text{LT50}(T)$ curve itself is the intermediate object both quantities are built from; it lives in `et_ext$lt50_curve` (the output of `derive_ltx_curve()`) and can be plotted directly (Figure S6).

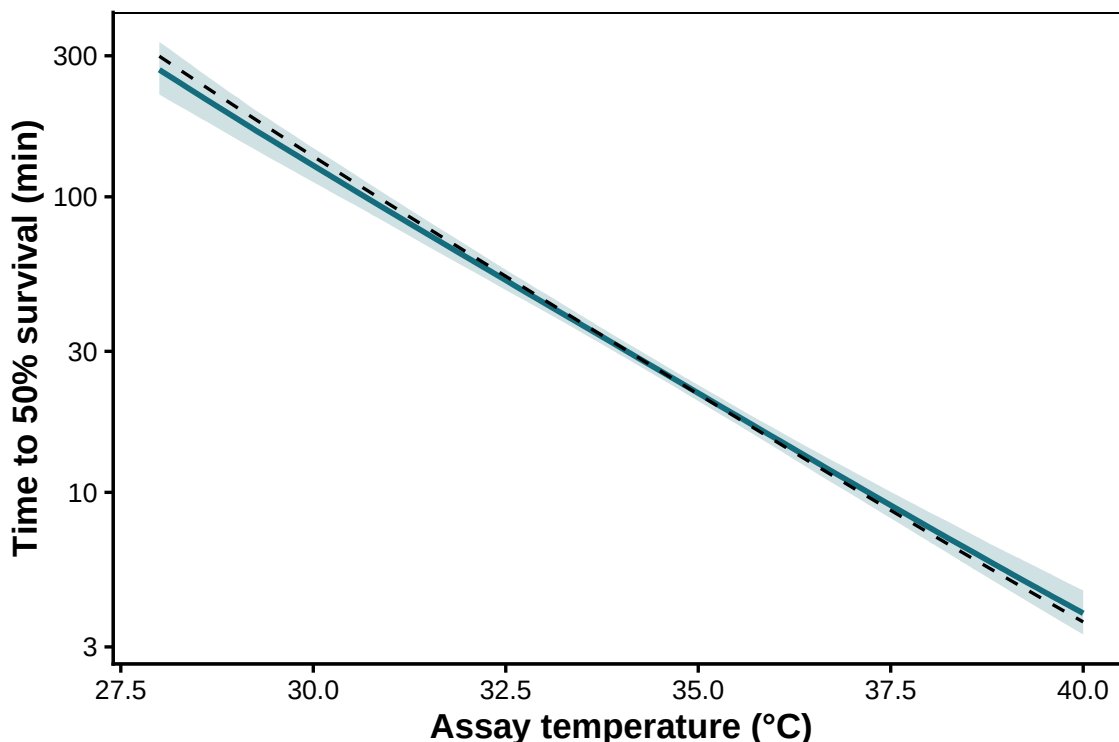


Figure S6: Posterior $\text{LT50}(T)$ curve (median + 95% CrI, blue) recovered by `derive_ltx_curve()` from the T-varying fit, with the analytical truth overlaid (dashed black). The curve is visibly bent rather than straight in T , and the posterior tracks the truth.

We can also drop down to the per-draw level and compute a *local* $z(T)$ at each assay temperature

from the same `et_ext$lt50_curve` posterior — a finite difference of $\log_{10}$ LT50 around T per draw, summarised across draws (Figure S7).

```
# Finite-difference local z at each assay T, per posterior draw, derived
# from et_ext$lt50_curve (the same per-draw LT50(T) curve extract_tdt
# already computed for the OLS).
half_step <- 0.5

local_z_summary <- purrr::map_dfr(temps, function(T_at) {
  z_per_draw <- et_ext$lt50_curve$draws |>
    dplyr::group_by(.draw) |>
    dplyr::summarise(
      z_local = -1 / ((stats::approx(temp, log10(duration_out),
                                     xout = T_at + half_step)$y -
                                     stats::approx(temp, log10(duration_out),
                                     xout = T_at - half_step)$y) /
                    (2 * half_step)),
      .groups = "drop"
    )
  q <- stats::quantile(z_per_draw$z_local, c(0.025, 0.5, 0.975),
                      na.rm = TRUE, names = FALSE)
  tibble::tibble(T_at = T_at,
                 z_local_median = q[2],
                 z_local_lower = q[1],
                 z_local_upper = q[3])
})

local_z_summary |>
  dplyr::left_join(
    tibble::tibble(T_at = temps, z_local_truth = truth_ext$local_z_true),
    by = "T_at"
  ) |>
  dplyr::select(T_at, z_local_truth, z_local_median,
                z_local_lower, z_local_upper) |>
  dplyr::rename(
    `T (°C)` = T_at,
    `z local truth (°C)` = z_local_truth,
    Median = z_local_median,
    Lower = z_local_lower,
    Upper = z_local_upper
  ) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

T (°C)	z local truth (°C)	Median	Lower	Upper
30	6.05	6.33	5.72	6.97
32	6.22	6.5	6.04	7.05
34	6.27	6.53	6.08	7.07
36	6.34	6.57	6.08	7.16
38	6.53	6.75	6.21	7.39

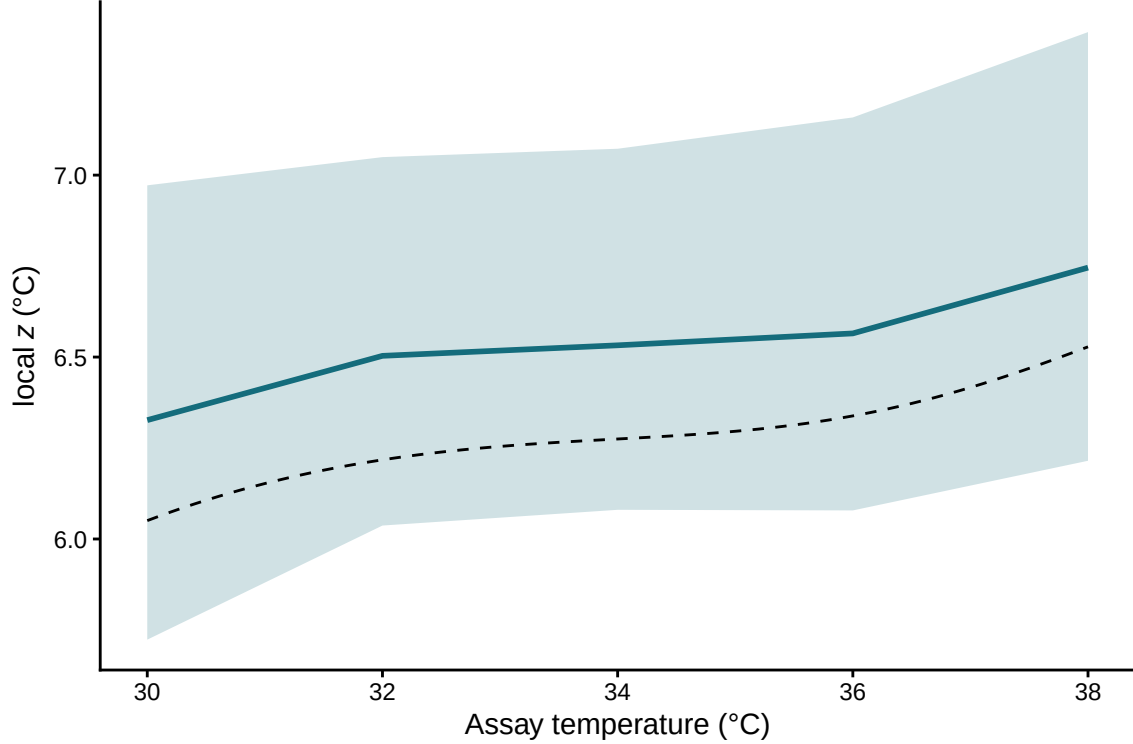


Figure S7: Local thermal sensitivity $z(T)$ recovered from the per-draw $\log_{10} \text{LT50}(T)$ curve (blue median + 95% CrI), with the analytical truth overlaid (dashed black). Where the constant-shape simple-case fit would give the same z at every assay temperature, here local z varies with T in lock-step with the bent $\log_{10} \text{LT50}(T)$ curve.

The headline result: the `same fit_4pl()` call and the `same extract_tdt()` call recover the bent $\log_{10} \text{LT50}(T)$ curve, the average z across the assay grid, and the per-draw $CT_{max_{1hr}}$ without modification. When the data carry real temperature signal on the shape parameters, the package picks it up; when the truth is constant-shape (as in Section 2.0.4) the same fit returns near-zero slopes on u , ℓ and k , the bent-curve calculation collapses to the closed-form one, and nothing is lost.

2.0.7 Heat injury accumulation and survival under a fluctuating trace

A particularly powerful aspect of TLS models is their ability to translate dynamic temperature time-series from nature and predict how such temperatures lead to heat injury (HI) accumulation

towards a threshold (e.g., LT_{50}) (Arnold *et al.* 2025; Jørgensen *et al.* 2021; Noble *et al.* 2026; Ørsted *et al.* 2022; Rezende *et al.* 2020). The joint Bayesian 4PL fit gives us not just point estimates of z and $CT_{max_{1hr}}$ but their full joint posterior, which we can push through the HI integral to obtain a posterior distribution over the predicted HI trajectory. We can also use the *full* 4PL surface to predict the cumulative survival fraction $S_{pred}(t)$ across the trajectory (Equation 9 of the manuscript). The library function `predict_heat_injury()` does both calculations in a single call and returns both trajectories with full posterior uncertainty.

2.0.7.1 A simulated temperature profile across 5 days

To mimic conditions a moderately heat-tolerant ectotherm might experience, we generate 5 days of hourly temperatures with a sinusoidal diurnal cycle, ranging from 12 °C overnight to a midday peak of 24 °C. The critical temperature T_c — below which damage is repaired as fast as it accrues and so there is no net damage increase (Jørgensen *et al.* 2021; Ørsted *et al.* 2022) — is set to 20 °C. The simulated organism has $CT_{max_{1hr}} \approx 32.1^\circ\text{C}$ and $z \approx 6.7^\circ\text{C}$, so the daily peak at 24 °C remains 8 °C below the 1-hour critical limit — moderate, sub-lethal stress that builds cumulatively.

```
hi_trace <- tibble::tibble(
  time_h = 0:(24 * 5 - 1),
  temp    = 18 + 6 * sin(2 * pi * (0:(24 * 5 - 1) - 6) / 24) # range 12-24 °C
)
hi_T_c <- 20 # damage threshold (°C)
```

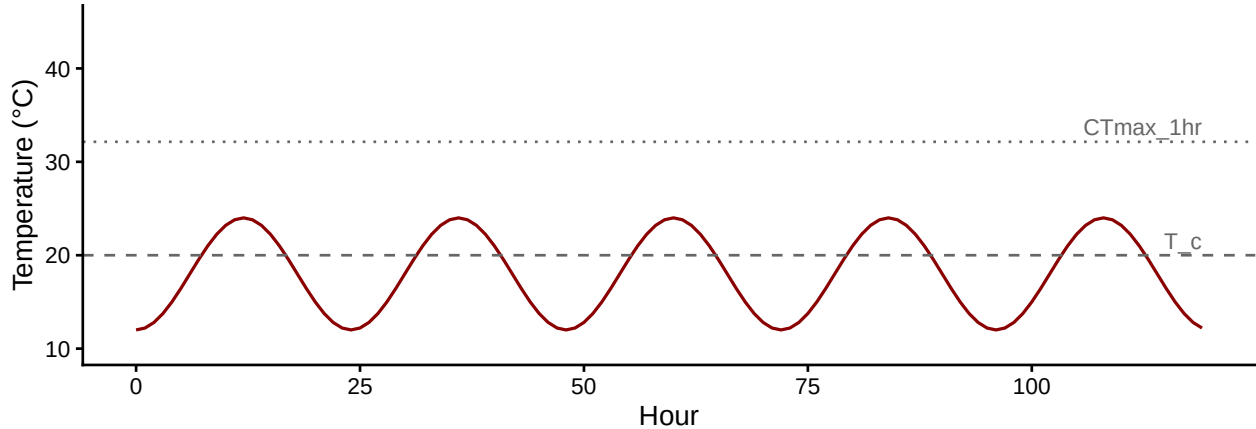


Figure S8: 5-day diurnal temperature trace. The dashed line is the damage-accumulation threshold $T_c = 20^\circ\text{C}$; the dotted line is the simulated organism’s $CT_{max_{1hr}} \approx 32.1^\circ\text{C}$ — well above the daily peaks, so accumulation is sub-lethal and gradual.

2.0.7.2 Posterior HI and survival via `predict_heat_injury()`

`predict_heat_injury()` takes a temperature trace plus a fitted workflow, propagates the full posterior of $(\ell, u, k, \beta_0, \beta_1)$ through the HI integral, and returns the median and 95% credible band of cumulative HI and predicted survival at every time point. We supply T_c to enforce zero damage accumulation below the threshold (matching Equation 7 of the manuscript). The model time units

here are minutes (because `standardize_data()` was called with `duration_unit = "minutes"`), so the trace's `time_h` is in hours and damage rates are computed accordingly internally.

```
# Convert hourly trace to minutes to match the model's time units.
hi_trace_min <- dplyr::mutate(hi_trace, time_h = time_h * 60)

hi <- predict_heat_injury(
  trace      = hi_trace_min,
  workflow   = wf_bb,
  target_surv = 0.5,
  T_c        = hi_T_c,
  ndraws     = 500
)
```

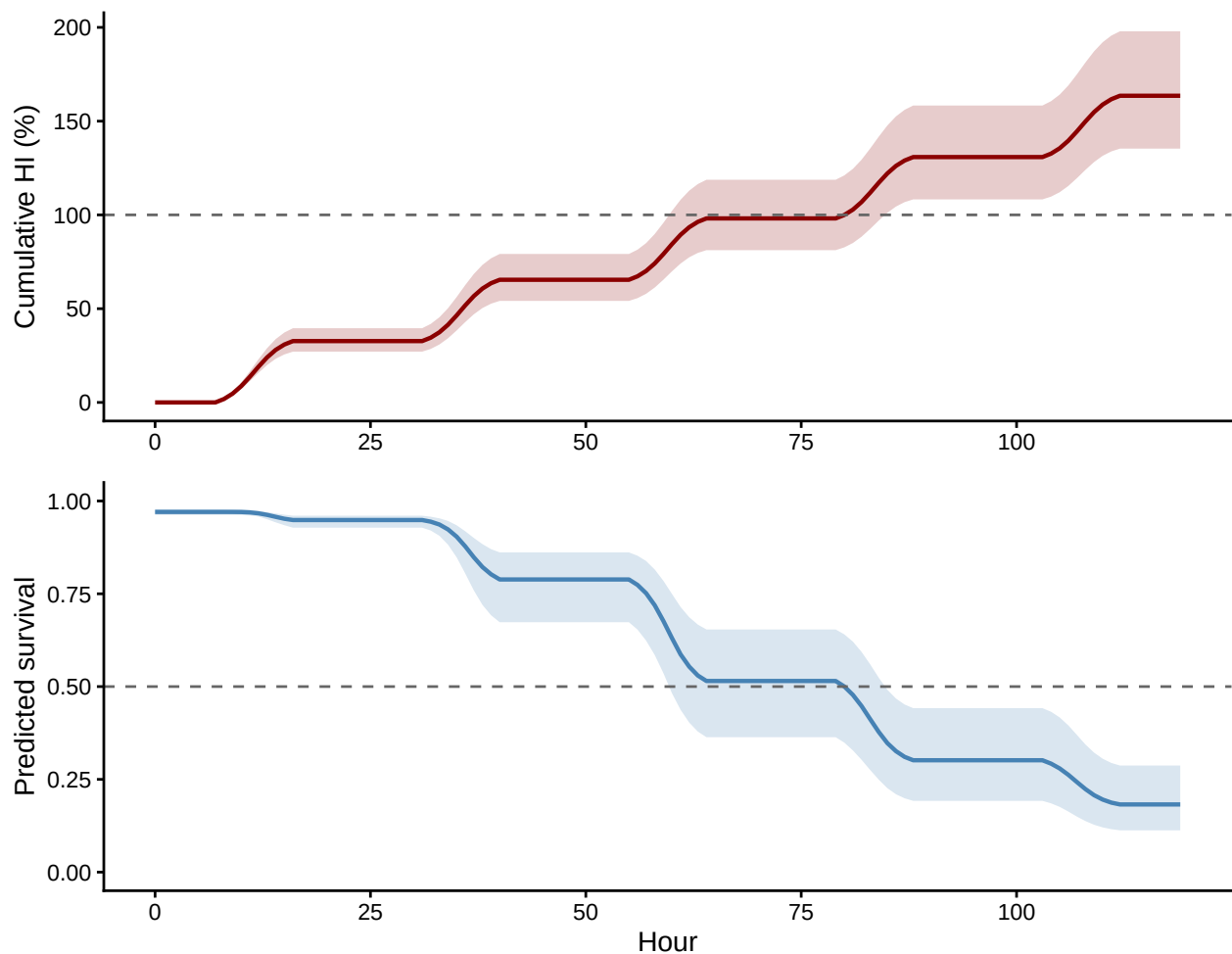


Figure S9: Posterior heat-injury accumulation (top, red) and predicted survival fraction (bottom, blue) under the 5-day field trace. Solid lines are posterior medians across 500 draws from `wf_bb`; shaded bands are 95% credible intervals. The dashed lines mark the LT50 threshold ($HI = 100\%$) and 50% survival respectively. The two trajectories tell the same story: $S_{\text{pred}}(t)$ falls through 0.5 at the same time $HI(t)$ crosses 100%.

By the end of day 5 the posterior median HI is 164% (95% CI: 135% to 198%), and median predicted survival is 0.18 (95% CI: 0.11 to 0.29).

2.0.8 Validation: recovering known planted doses

Before trusting `predict_heat_injury()` on real field data, we validate it against analytical truth on three reference traces with known dosing structure. `make_temperature_scenarios()` builds the traces; `planted_dose_from_trace()` implements the classical HI integral (Equation 7) directly given known z and CT_{max_1hr} . Posterior medians from `predict_heat_injury()` should sit near the analytical truth, and the 95% credible intervals should cover it.

```
# Three traces: flat baseline (zero HI), single hourly spike, three spikes.
val_scens <- make_temperature_scenarios(
  baseline      = 20,
  spike_temp    = 28,
  n_hours       = 96,
  spike_times_single = 24,
  spike_times_multi  = c(24, 48, 72)
)
val_T_c <- 24

# Analytical truth, given the simulation parameters (true_z, true_CTmax).
planted <- purrr::map(val_scens, planted_dose_from_trace,
  z = true_z, CTmax_1hr = true_CTmax, T_c = val_T_c)

# Posterior predictions from the fitted beta-binomial workflow. The model
# expects minutes; convert each trace's time axis.
predicted <- purrr::map(val_scens, function(sc) {
  predict_heat_injury(
    trace      = dplyr::mutate(sc, time_h = time_h * 60),
    workflow   = wf_bb,
    target_surv = 0.5,
    T_c        = val_T_c,
    ndraws     = 300
  )
})

# Final HI per scenario: analytical truth vs posterior median + CrI.
val_table <- tibble::tibble(
  Scenario      = c("Flat", "Single spike", "Multi spike"),
  `Planted HI (%)` = c(tail(planted$flat$hi_cumulative, 1),
    tail(planted$single_spike$hi_cumulative, 1),
    tail(planted$multi_spike$hi_cumulative, 1)),
  `Posterior median (%)` = c(tail(predicted$flat$summary$hi_median, 1),
    tail(predicted$single_spike$summary$hi_median, 1),
    tail(predicted$multi_spike$summary$hi_median, 1)),
  `Lower (%)` = c(tail(predicted$flat$summary$hi_lower, 1),
```

Scenario	Planted HI (%)	Posterior median (%)	Lower (%)	Upper (%)	Truth in CrI?
Flat	0	0	0	0	TRUE
Single spike	24	22	20	24	TRUE
Multi spike	72	67	60	73	TRUE

```

        tail(predicted$single_spike$summary$hi_lower, 1),
        tail(predicted$multi_spike$summary$hi_lower, 1)),
`Upper (%)` = c(tail(predicted$flat$summary$hi_upper, 1),
        tail(predicted$single_spike$summary$hi_upper, 1),
        tail(predicted$multi_spike$summary$hi_upper, 1))
)
val_table$`Truth in CrI?` <- with(val_table,
        `Lower (%)` <= `Planted HI (%)` &
        `Planted HI (%)` <= `Upper (%)`)
tinytable::tt(val_table, digits = 2, escape = TRUE)

```

The flat trace stays below T_c for its entire duration and so accrues zero HI, matching the analytical truth exactly. The single- and multi-spike traces deliver known doses analytically; `predict_heat_injury()` recovers each with the truth lying inside its credible interval. This sanity-check pattern — three traces with known doses, plus the recovery check — is what each new dataset’s heat-injury prediction should pass before being trusted.

3 Case Study 1: Shrimp data

We now apply the full workflow described above — `standardize_data()` → `fit_4pl()` → `extract_tdt()` → `predict_heat_injury()` — to lethal-TDT data from brown shrimp (*Crangon crangon*). The data are 148 replicate trials spanning 30–33 °C and 5 min to 6 hours of exposure, with each replicate exposing a tank of individuals to a fixed temperature × duration combination on a specific date. The two grouping factors **Date** and **Tank** enter the model as random intercepts on `mid`, allowing day-to-day batch variation and tank-level cohort variation to be partitioned out of the dose-response surface.

3.1 Data and standardisation

```

shrimp_raw <- readxl::read_excel(
  here::here("data", "Data_Shrimp_TDT_CTmax_experiment.xlsx"),
  sheet = "LETHAL_TDT"
)

shrimp_std <- standardize_data(
  data      = shrimp_raw,
  temp      = "Temperature_assay",

```

Table S9: Summary of the shrimp lethal-TDT dataset after standardisation. Temperature centre is the grand mean used by the model for `temp_c`.

n trials	Temperature range (°C)	Duration range (hr)	Median individuals/trial	Temperature centre (°C)
148	30–33	0.08–6	10	32

Table S10: Sampling diagnostics for the shrimp 4PL fit. Healthy targets: $R_{hat} < 1.01$, $ESS > 400$, zero divergences.

rhat_max	ess_bulk_min	ess_tail_min	divergences	treedepth_hits	rhat_pass	ess_pass	divergence_
1	1468	2374	0	2	TRUE	TRUE	TRUE

```

duration      = "Duration_exposure_hours",
n_total       = "N_individuals_after_trial",
mortality     = "Mortality_after_trial",
random_effects = c("Date", "Tank"),
duration_unit  = "hours"
)

```

3.2 Fitting the joint Bayesian 4PL

We fit the default `fit_4pl()` model — beta-binomial likelihood, identity link, the disjoint-bounds asymptote reparameterisation, and `temp_c` slopes on all four 4PL sub-parameters — with random intercepts on `mid` for `Date` and `Tank`. The fit takes a few minutes on first render; subsequent renders reload it from disk because `file_refit = "on_change"`.

```

wf_shrimp <- fit_4pl(
  shrimp_std,
  random_effects = c("Date", "Tank"),
  chains = 4, iter = 3000, warmup = 1500, cores = 4, seed = 123,
  control = list(adapt_delta = 0.99, max_treedepth = 12),
  file     = file.path(models_dir, "fit_shrimp_lethal_4pl")
)

```

3.2.1 Sampling diagnostics

3.2.2 Posterior parameters on the natural scale

3.3 Survival curves with observed overlay

We predict survival probability across the observed temperature $\times$ duration grid and overlay the observed proportions per replicate. The fitted curves should track the observed clouds at every temperature.

Table S11: Posterior medians and 95% credible intervals for the shrimp 4PL fit, transformed back to the natural scale. **low** and **up** are the bottom and top asymptotes of the survival curve at the grand-mean temperature; **k** is the slope on $\log_{10}$ time; **mid** intercept is $\log_{10}$ LT50 in hours at the grand-mean temperature; **mid temp_c** slope is β_1 , from which $z = -1/\beta_1$.

parameter	median	lower	upper
low (lower asymptote)	0.00692	0.00228	0.0222
up (upper asymptote)	0.95174	0.89351	0.9928
k (slope)	5.82927	4.68871	7.4851
mid intercept (at T_bar)	0.05697	-0.30812	0.4094
mid temp_c slope	-0.36949	-0.42841	-0.3109
z (°C)	2.70643	2.33424	3.2163

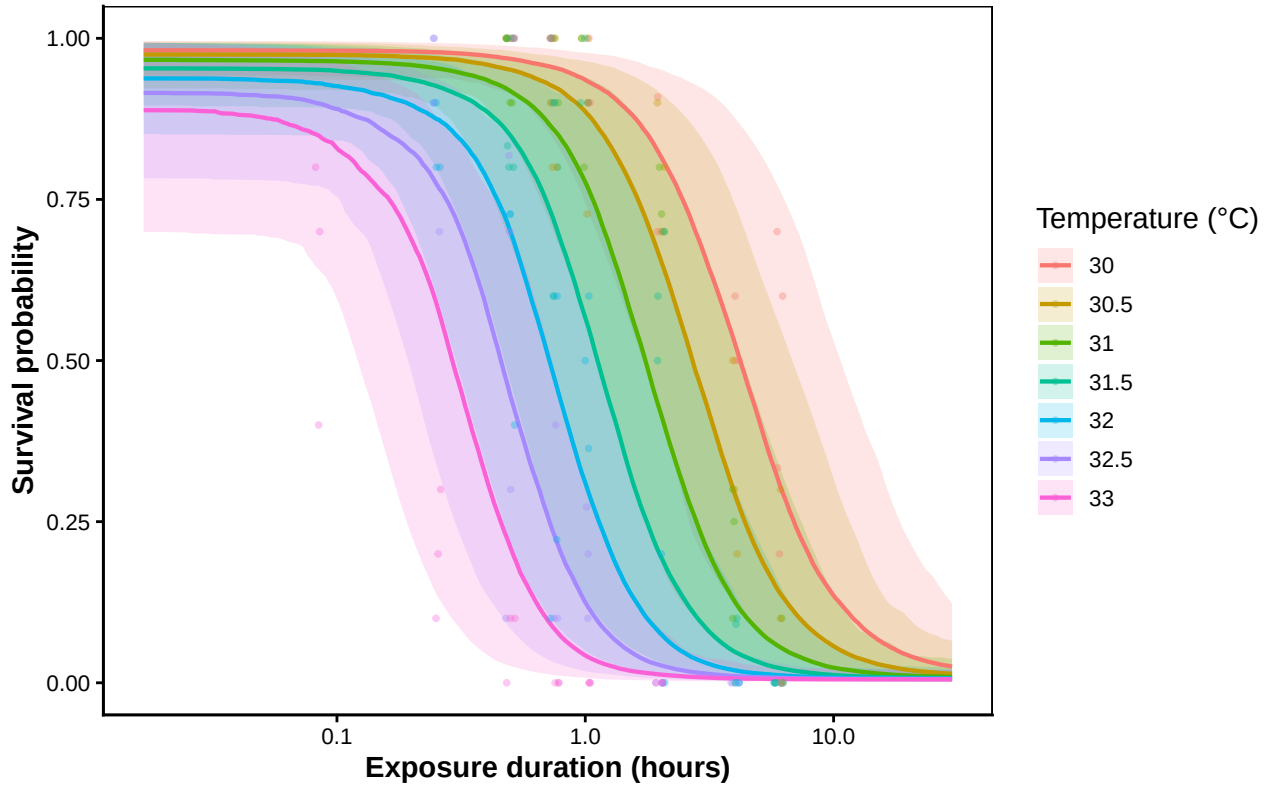


Figure S10: Posterior survival curves for brown shrimp at each assayed temperature, with 95% credible bands. Coloured points are observed per-replicate survival proportions. Duration on the log scale; population-level prediction (random effects marginalised).

3.4 TDT landscape

A dense 2-D view of survival across the entire (temperature, duration) plane shows the dose-response surface as a heatmap with overlaid contours at 25, 50, and 75% survival.

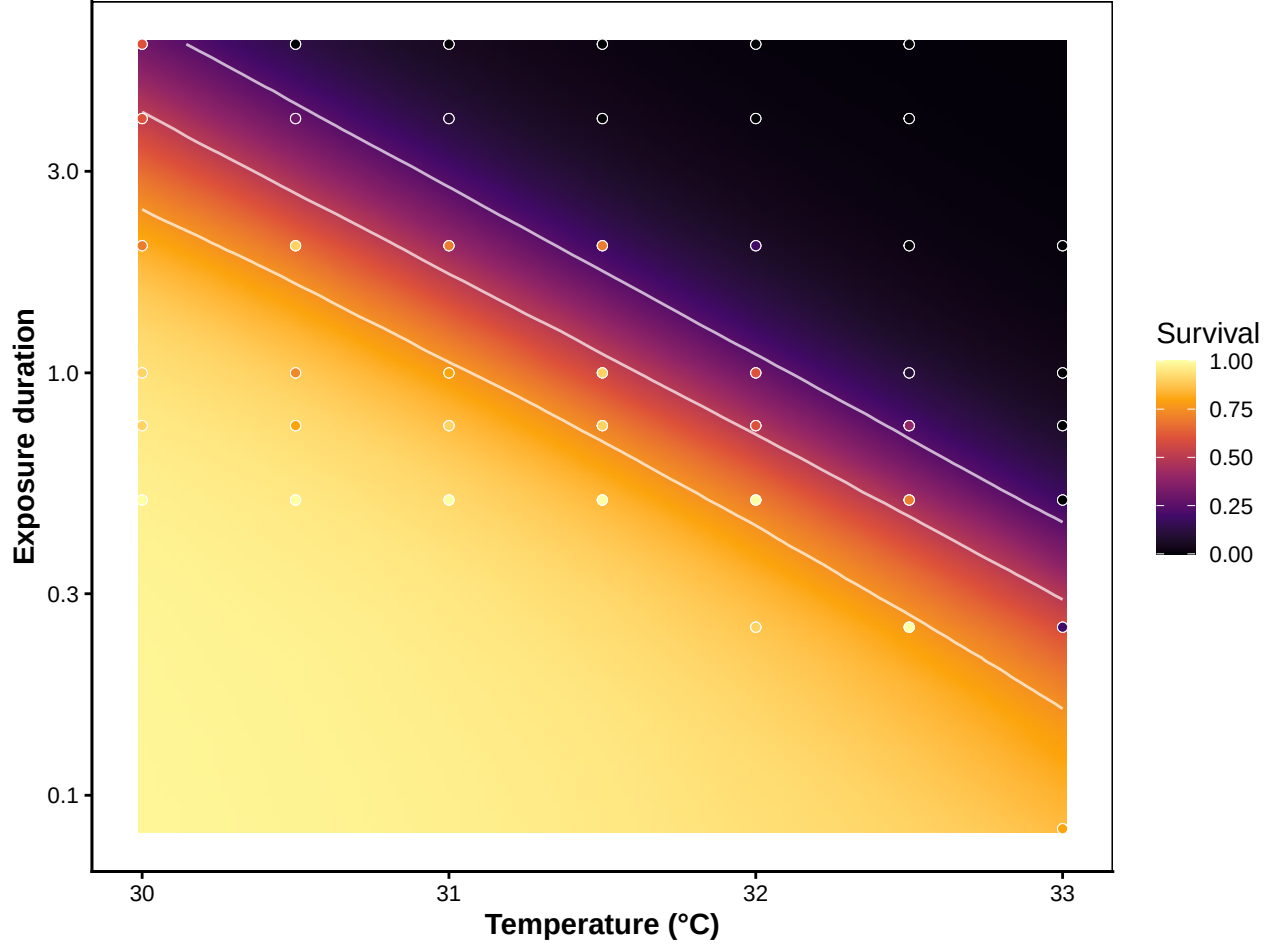


Figure S11: TDT landscape for brown shrimp: posterior median predicted survival across a 120×120 grid of assay temperatures $\times$ durations. White contours mark 25%, 50%, and 75% survival isolines. Overlaid points are raw observed proportions.

3.5 Classical TDT quantities: z , $CT_{max_{1hr}}$, T_{crit}

`extract_tdt()` bundles z (from per-draw log-linear regression of the LT50 curve), $CT_{max_{1hr}}$ (4PL inversion at the 60-minute reference for 50% survival), and T_{crit} (the rate-multiplier damage-threshold, integrated over $r^* \in [0.1, 1]$ % HI per hour — see Equation S3) in a single call. Every output inherits the model's posterior, so all three quantities come with calibrated credible intervals.

```
et_shrimp <- extract_tdt(
  wf_shrimp,
  t_ref      = 60,          # CTmax defined at 1-hour exposure
  TC_rate_range = c(0.1, 1), # T_crit integrated over r* in % HI/hr
  ndraws     = 500
)
```

Table S12: Classical TDT summaries for brown shrimp, with full posterior uncertainty. z comes from a per-draw log-linear regression of $\log_{10}$ LT50 on temperature. $CT_{max_{1hr}}$ is the temperature at 50% survival after 1-hour exposure (4PL inversion per draw). T_{crit} is the temperature at which the TDT damage rate falls below r^* % HI per hour, with r^* sampled uniformly on $\log_{10}$ across $[0.1, 1]$ — the pooled posterior carries both parameter uncertainty and operational uncertainty in r^* .

Quantity	Median	Lower	Upper
z ($^{\circ}\text{C}$ per decade of time)	2.6	2.3	3
CT_{max_1hr} ($^{\circ}\text{C}$)	31.6	30.6	33
T_{crit} ($^{\circ}\text{C}$, r^* in $[0.1, 1]$ per hour)	25.1	23	27

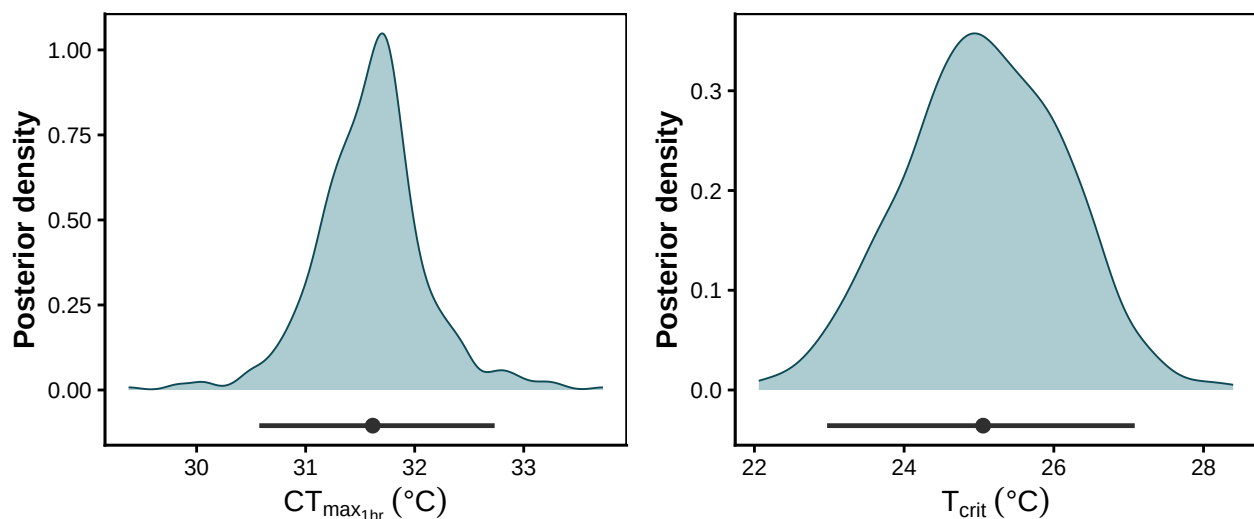


Figure S12: Posterior densities of $CT_{max_{1hr}}$ (left) and T_{crit} (right) for brown shrimp. The horizontal bar below each density is the 95% credible interval; the point marks the posterior median. The 5–6 $^{\circ}\text{C}$ gap between the two quantities represents the sub-lethal window between the onset of detectable injury and the LT50 limit.

3.6 Heat injury under three reference traces

We now project the fitted shrimp model under the same three-scenario harness used in the simulation validation above (§[Validation: recovering known planted doses](#)). The traces are calibrated to shrimp ecology and the posterior of the fitted model: 17 $^{\circ}\text{C}$ is the natural holding temperature, 30 $^{\circ}\text{C}$ is a heat-stress spike that sits between T_{crit} and $CT_{max_{1hr}}$, and the damage threshold passed to `predict_heat_injury()` is the posterior median of T_{crit} from `extract_tdt()` — model-derived rather than hand-picked.

```
# Damage threshold = posterior median of T_crit from the fit. Model-derived.
shrimp_T_c <- et_shrimp$T_crit$summary$temp_median

shrimp_scens <- make_temperature_scenarios(
```

```

baseline    = 17,
spike_temp  = 30,      # between T_crit and CTmax_1hr - clearly stressful
n_hours     = 96,
spike_times_single = 24,
spike_times_multi  = c(24, 48, 72)
)

# Predict HI + survival under each scenario, without repair.
shrimp_hi <- purrr::map(shrimp_scens, function(sc) {
  predict_heat_injury(
    trace      = sc,
    workflow   = wf_shrimp,
    target_surv = 0.5,
    T_c        = shrimp_T_c,
    ndraws     = 500
  )
})

# Sharpe-Schoolfield repair parameters tuned to shrimp: optimum ~17 °C, modest
# repair capacity. r_ref is in (doses per hour) because the model time unit
# for the shrimp fit is hours.
shrimp_repair_pars <- list(
  TA    = 14065,
  TAL   = 50000,
  TAH   = 120000,
  TL    = 10.5 + 273.15,
  TH    = 22.5 + 273.15,
  TREF  = 17 + 273.15,
  r_ref = 0.005 # 0.5% LT50-dose repaired per hour at TREF
)

# Re-run with repair on the multi-spike scenario to show the offsetting effect.
shrimp_hi_repair <- predict_heat_injury(
  trace      = shrimp_scens$multi_spike,
  workflow   = wf_shrimp,
  target_surv = 0.5,
  T_c        = shrimp_T_c,
  ndraws     = 500,
  repair     = TRUE,
  repair_pars = shrimp_repair_pars
)

```

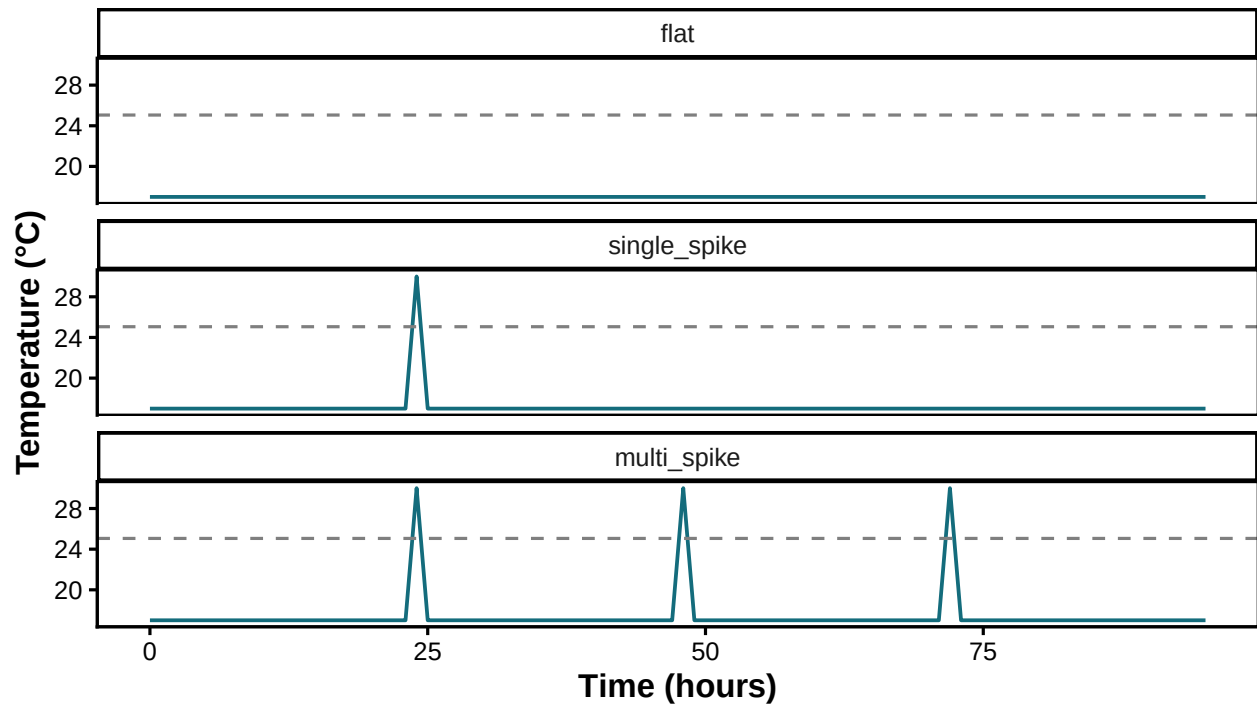


Figure S13: The three reference temperature traces used for the shrimp heat-injury prediction. Baseline 17 °C (shrimp natural holding temperature); 1-hour spikes at 28 °C; the dashed line marks the damage threshold T_c set at the lower 95% credible bound of T_{crit} .

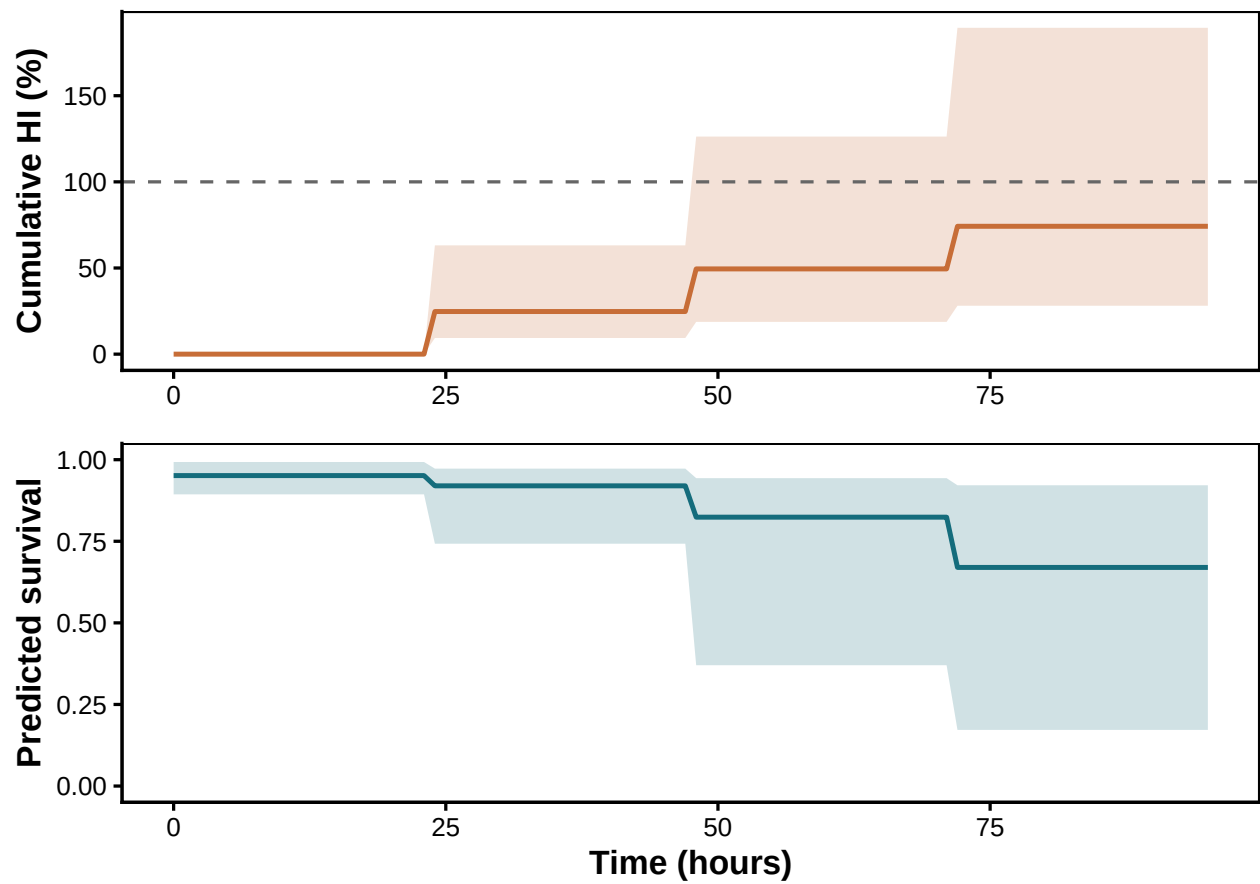


Figure S14: Cumulative heat injury (top) and predicted survival (bottom) for brown shrimp under the multi-spike trace, **no repair**. Posterior median lines and 95% credible bands.

Table S13: Final cumulative HI and predicted survival under each scenario for brown shrimp, posterior medians with 95% credible intervals.

Scenario	Repair	HI median (%)	HI lower	HI upper	Survival median	Survival lower	Survival upper
Flat	None	0.0	0.0	0.0	0.950	0.898	1.000
Single spike	None	25.6	10.7	62.8	0.917	0.738	1.000
Multi spike	None	74.2	28.1	189.4	0.670	0.172	1.000
Multi spike	Sharpe-Schoolfield	45.6	0.7	172.0	0.780	0.217	1.000

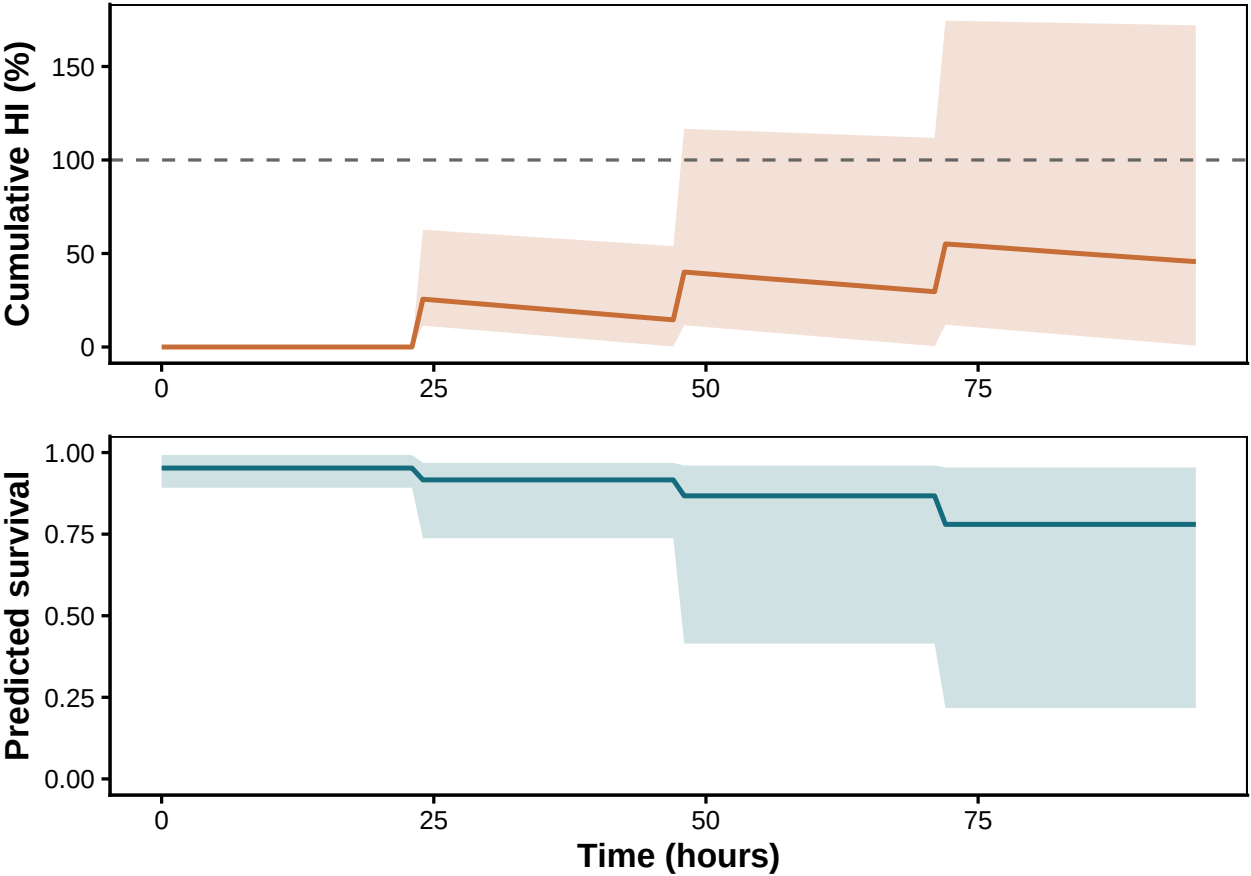


Figure S15: Same multi-spike trace as above, this time **with Sharpe-Schoolfield repair** active ($r_{\text{ref}} = 0.005$ doses per hour at 17 °C). Repair fully offsets the spike damage at sub- T_c baseline.

The shrimp case study shows the complete loop: raw counts $\rightarrow$ fitted model $\rightarrow$ classical TDT quantities (with credible intervals on every one of them) $\rightarrow$ heat-injury predictions under fluctuating field traces (with and without repair). Every quantity reported here inherits the joint posterior of the single 4PL fit — there is no second-stage regression, no point-estimate-then-bootstrap, no separate ramping-rate experiment. The same machinery applies to any TDT-style proportion dataset by changing only `standardize_data()`'s column-name arguments.

4 References

- Arnold, P.A., Noble, D.W.A., Nicotra, A.B., Kearney, M.R., Rezende, E.L., Andrew, S.C., *et al.* (2025). [A framework for modelling thermal load sensitivity across life](#). *Global Change Biology*, 31, e70315.
- Jørgensen, L.B., Malte, H. & Overgaard, J. (2021). [A unifying model to estimate thermal tolerance limits in ectotherms across static, dynamic and fluctuating exposures to thermal stress](#). *Scientific Reports*, 11, 12840.
- Noble, D.W.A., Mayfield, M.M., Hoffmann, A.A., Chen, Z.-H., Lade, S.J., Bai, X., *et al.* (2026). [A systems modelling approach to predict biological responses to extreme heat](#). *Trends in Ecology & Evolution*, 41, 451–464.
- Ørsted, M., Jørgensen, L.B. & Overgaard, J. (2022). [Finding the right thermal limit: A framework to reconcile ecological, physiological and methodological aspects of CTmax in ectotherms](#). *Journal of Experimental Biology*, 225, jeb244514.
- Rezende, E.L., Bozinovic, F., Szilágyi, A. & Santos, M. (2020). [Predicting temperature mortality and selection in natural *Drosophila* populations](#). *Science*, 369, 1242–1245.
- Rezende, E.L., Castañeda, L.E. & Santos, M. (2014). [Tolerance landscapes in thermal ecology](#). *Functional Ecology*, 28, 799–809.

5 Session Information

R version 4.5.3 (2026-03-11)

Platform: aarch64-apple-darwin20

locale: C.UTF-8|C.UTF-8|C.UTF-8|C|C.UTF-8|C.UTF-8

attached base packages: *stats*, *graphics*, *grDevices*, *utils*, *datasets*, *methods* and *base*

other attached packages: *bayesTLS*(v.0.1.0), *readxl*(v.1.4.5), *pander*(v.0.6.6), *patchwork*(v.1.3.1), *tinytable*(v.0.16.0), *tidybayes*(v.3.0.7), *posterior*(v.1.6.1), *brms*(v.2.23.0), *Rcpp*(v.1.1.1-1.1), *ggplot2*(v.4.0.3), *purrr*(v.1.2.2), *tibble*(v.3.3.1), *tidyr*(v.1.3.2), *dplyr*(v.1.2.1) and *here*(v.1.0.1)

loaded via a namespace (and not attached): *svUnit*(v.1.0.8), *tidyselect*(v.1.2.1), *viridis-Lite*(v.0.4.3), *farver*(v.2.1.2), *loo*(v.2.9.0), *S7*(v.0.2.2), *fastmap*(v.1.2.0), *TH.data*(v.1.1-3), *tensorA*(v.0.36.2.1), *pacman*(v.0.5.1), *digest*(v.0.6.39), *estimability*(v.1.5.1), *lifecycle*(v.1.0.5), *StanHeaders*(v.2.32.10), *survival*(v.3.8-6), *processx*(v.3.9.0), *magrittr*(v.2.0.5), *compiler*(v.4.5.3), *rlang*(v.1.2.0), *tools*(v.4.5.3), *yaml*(v.2.3.12), *knitr*(v.1.51), *labeling*(v.0.4.3), *bridgesampling*(v.1.2-1), *curl*(v.7.1.0), *pkgbuild*(v.1.4.8), *plyr*(v.1.8.9), *RColorBrewer*(v.1.1-3), *cmdstanr*(v.0.9.0), *multcomp*(v.1.4-28), *abind*(v.1.4-8), *withr*(v.3.0.2), *grid*(v.4.5.3), *stats4*(v.4.5.3), *colorspace*(v.2.1-1), *xtable*(v.1.8-8), *inline*(v.0.3.21), *emmeans*(v.2.0.3), *scales*(v.1.4.0), *MASS*(v.7.3-65), *isoband*(v.0.3.0), *tinytex*(v.0.59), *cli*(v.3.6.6), *mvtnorm*(v.1.3-7), *rmarkdown*(v.2.31), *generics*(v.0.1.4), *otel*(v.0.2.0), *RcppParallel*(v.5.1.11-2), *reshape2*(v.1.4.5), *rstan*(v.2.32.7), *stringr*(v.1.6.0), *splines*(v.4.5.3), *bayesplot*(v.1.15.0), *parallel*(v.4.5.3), *cellranger*(v.1.1.0), *matrixStats*(v.1.5.0), *vc-trs*(v.0.7.3), *V8*(v.6.0.4), *Matrix*(v.1.7-4), *sandwich*(v.3.1-1), *jsonlite*(v.2.0.0), *callr*(v.3.7.6), *litedown*(v.0.7), *arrayhelpers*(v.1.1-0), *ggdist*(v.3.3.3), *glue*(v.1.8.1), *codetools*(v.0.2-20), *distributional*(v.0.6.0), *stringi*(v.1.8.7), *gtable*(v.0.3.6), *QuickJSR*(v.1.9.0), *pillar*(v.1.11.1), *htmltools*(v.0.5.9), *Brodingnag*(v.1.2-9), *R6*(v.2.6.1), *rprojroot*(v.2.1.1), *evaluate*(v.1.0.5),

lattice(v.0.22-9), *backports(v.1.5.1)*, *rstantools(v.2.6.0)*, *gridExtra(v.2.3)*, *coda(v.0.19-4.1)*,
nlme(v.3.1-168), *checkmate(v.2.3.4)*, *mgcv(v.1.9-4)*, *xfun(v.0.57)*, *zoo(v.1.8-14)* and *pkgcon-*
fig(v.2.0.3)

...